Cluster Computing (2026) 29:309
https://doi.org/10.1007/s10586-026-06095-9

Anomaly detection and root-cause identification in microservices: a survey

Luís M. Barata[1,2] · Sérgio Sequeira[3] · Eurico Lopes[2] · Pedro R. M. Inácio[1] · Mário M. Freire[1,4]

Received: 14 June 2025 / Revised: 2 March 2026 / Accepted: 4 March 2026

Abstract
Microservices Architecture (MSA) has emerged as a dominant paradigm for building scalable and flexible distributed systems. By decomposing applications into loosely coupled services, MSA improves modularity and deployment agility, but also introduces significant challenges for system monitoring, anomaly detection, and root-cause identification. These challenges stem from the high degree of dynamic behaviour, service heterogeneity, and inter-service dependencies. This survey provides a comprehensive and systematic review of state-of-the-art approaches for anomaly detection and root-cause analysis in microservice-based systems, covering the period from 2012 to 2025. It examines 117 selected studies, categorized according to data collection methods, detection algorithms, diagnostic models, and evaluation metrics. Emphasis is placed on the role of machine learning, statistical inference, and trace-based methods in detecting and localizing faults. Furthermore, the survey highlights the limitations of existing techniques in terms of scalability, explainability, and real-time applicability. By consolidating scattered research into a unified analytical framework, this work contributes a detailed taxonomy of methods and a critical discussion of their effectiveness in container-based virtualized environments. The survey provides a representative and timely overview of the field, helping to map current progress and highlight remaining gaps. Finally, it outlines promising directions such as hybrid detection architectures, automated trace reasoning, and context-aware root-cause localization, offering guidance for researchers and practitioners seeking to enhance reliability, observability, and operational resilience in microservice systems.

Keywords Anomaly Detection · Data Analysis · Data Collection Methods · Machine Learning · Microservices · Root-Cause Identification

1 Introduction

The rise of cloud computing has enabled providers to deliver Infrastructure as a Service (IaaS) and Platform as a Service (PaaS), supported by Microservices Architecture (MSAs) where monolithic applications are divided into smaller services distributed across containers [1, 2]. Container-based virtualization underpins modern MSAs by encapsulating code and dependencies into lightweight, portable units that share the same kernel, avoiding the overhead of full OS

✉ Luís M. Barata
luis.miguel.barata@ubi.pt

Sérgio Sequeira
sergio.sequeira@ubi.pt

Eurico Lopes
eurico@ipcb.pt

Pedro R. M. Inácio
prmi@ubi.pt

Mário M. Freire
mario.freire@ubi.pt

[1] Instituto de Telecomunicações, Department of Computer Science, Universidade da Beira Interior, Rua Marquês D'Ávila e Bolama, 6201-001 Covilhã, Portugal

[2] Department of Computer Science, Escola Superior de Tecnologia, Instituto Politécnico de Castelo Branco, Av. do Empresário, 6000-767 Castelo Branco, Portugal

[3] Department of Computer Science, Universidade da Beira Interior, Rua Marquês D'Ávila e Bolama, 6201-001 Covilhã, Portugal

[4] NOVA Laboratory for Computer Science and Informatics (NOVA LINCS), Department of Computer Science, Universidade da Beira Interior, Covilhã, Portugal

Published online: 03 June 2026

virtualization [3, 4]. This approach enhances scalability, portability, and cost efficiency, and is now the foundation of most cloud-native deployments [5, 6]. Leading platforms such as Amazon, Netflix, eBay, LinkedIn, and Alibaba rely on containerized microservices to operate at massive scale, with Alibaba alone managing more than 30,000 services [7].

Both academia and industry have invested heavily in monitoring solutions. Tools such as Amazon CloudWatch, Azure CloudMonix, and OpenStack Ceilometer offer basic monitoring, while resilience-testing frameworks like Netflix Chaos Monkey deliberately terminate instances to validate recovery [8, 9]. However, these tools are insufficient for highly dynamic microservice environments, where a single user request can trigger thousands of backend calls [10]. Large-scale ecosystems, such as those of Alibaba and JD.com, orchestrate tens of thousands of services using Kubernetes and Istio [11], making effective anomaly detection a critical challenge.

Although outlier detection has been studied since the 19th century [12], there is still no universal solution for modern distributed systems [13]. Existing methods suffer from several constraints: many require manual selection of monitored metrics and alert policies, others can detect suspicious metrics but fail to identify root causes due to the granularity of microservice environments, and some rely on execution traces that demand expert intervention [14–16]. In practice, these shortcomings often result in severe disruptions. Amazon, for instance, lost millions of dollars during outages in 2013 and 2018 [17, 18], while Walmart's monolithic e-commerce architecture repeatedly collapsed during peak events until it migrated to microservices, improving resilience, conversion rates, and cost efficiency [19]. As services may scale from a few to thousands of instances across dynamic infrastructures, even minor faults such as misconfiguration, hardware failure, or queue overload can propagate rapidly along service call chains [7, 20]. Detecting unusual behaviour before it escalates is therefore essential to safeguard availability and prevent cascading failures.

Beyond detection accuracy, recent research emphasizes the importance of trustworthy diagnosis in distributed systems. In microservice environments, anomaly detection and root-cause identification must be reliable, explainable, and consistent despite noisy and incomplete telemetry. This requirement motivates a shift from traditional Trusted AI toward Trusted Distributed AI (TDAI) [28–30], where trust must be established not only at the model level but across distributed data, causal dependencies, and system-level behavior. This survey therefore analyses existing approaches not only from a technological perspective but also through the lens of trustworthiness in distributed diagnostic systems.

This survey focuses on anomaly detection and root-cause (RC) identification within microservice environments and is organized as follows. Section 2 reviews similar survey works and outlines the research questions guiding this study. Section 3 provides a background on microservices and introduces the anomaly detection process in generic systems. Section 4 emphasizes anomaly detection and RC identification within microservice environments, elaborating on key techniques and approaches. Section 5 compares the performance of the surveyed methods and platforms, highlighting their relevance and applicability. Section 6 discusses challenges and open issues and outlines future research directions. Finally, Section 7 concludes the paper by summarizing the key findings and insights.

2 Related surveys and contribution

This section overviews and highlights key survey publications on anomaly detection and RC identification in microservices architecture systems, compiled in Table 1. This table also puts in evidence the contribution of this survey regarding previous surveys.

Only two survey publications [26, 27] directly related to our work were found in the most relevant digital libraries (IEEE Xplore, ACM Digital Library, Science Direct, Wiley Online Library, Springer Link, Scopus, and Web of Science). Several other surveys can be found, but none is directly associated with anomaly detection in a microservice environment. These studies addressed both subjects separately. Research works presented by Chandola et al. [22] and Akoglu et al. [23] glanced at the anomaly detection process more globally and did not focus on microservices architecture. These studies were published before the formal definition of microservices and their global usage. Also, Steinder *et al.* [21], Wong *et al.* [24], and Solé *et al.* [25] described the process of root-cause identification but did not focus on the microservices architecture.

Soldani and Brogi [26] focused on microservices architecture. However, besides the topics addressed in our survey that were not considered in [26] (see Table 1), much research literature has been published in the last few months, so we also increased the value of the survey by including the latest publications on the subject. Regarding the survey presented by Zhang et al. [27], our survey discusses the performance and merit of the approaches, which were not addressed in [27], and updates the review with more recent publications.

Using the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) methodology, we conducted an extensive research process encompassing seven major online libraries, reviewing 1,700 results

Table 1 Surveys on (microservices) anomalies and root-cause identification

Author(s)	Year	Anomalies Detection	Root-cause Analysis	Anomalies Detection Algorithms	Root-causeAnalysis Algorithms	Types of Anomalies	Algorithms Performance	Available Datasets	Toolkits Platforms
Steinder and Sethi [21]	2004		✓		✓				
Chandola *et al.* [22]	2009	✓		✓					
Akoglu *et al.* [23]	2014	✓		✓					
Wong *et al.* [24]	2016		✓		✓				
Solé *et al.* [25]	2017		✓		✓				
Soldani and Brogi [26]	2021	✓	✓	✓	✓				
Zhang *et al.* [27]	2025	✓	✓	✓	✓	✓		✓	✓
This Survey	2026	✓	✓	✓	✓	✓	✓	✓	✓

and ultimately including 117 relevant publications. This approach minimizes the likelihood of excluding any significant studies on the topic.

With this survey article, we intend to analyze, discuss, and condense the results of our research on anomaly detection in microservices architecture, according to the topics presented in Table 1. This topic is important because microservice architectures are intricated and distributed, and these features may introduce new forms of anomaly inherent to these architectures.

We separated our findings into two significant parts: Anomaly Detection and Root-Cause Identification, intending to align our research work with the previously published works and those included in this survey. The following Research Questions (RQs) (Table 2) were formulated at the beginning of the investigation process, and this study later presents answers to these questions.

3 Background on microservices and anomaly detection

Before a literature review on Anomaly Detection in Microservices, it is essential to clarify the concept of anomalies, some types of anomalies, and how to deal with anomalies. We introduced microservices architecture to enlighten readers about the specifications of these types of architectures. In addition, it is essential to introduce methods to characterize the performance of various algorithms by explaining the most relevant and used metrics.

3.1 Definition of microservices

Despite being a relatively recent trend, the idea of dividing systems into independent services dates back to the UNIX operating system in 1978 [31]. Martin Fowler in 2015 defined *microservices* as " *an approach to developing a single application as a suite of small services, each running in its process and communicating with lightweight mechanisms, often an HTTP API"* [32].

Microservices build on principles of service-oriented architecture: system logic is decomposed into small, independent units that collaborate to provide full functionality [1]. This independence allows different technologies, programming languages, and databases, supporting multidisciplinary teams but also requiring careful coordination and governance. Communication and orchestration therefore remain key challenges.

Today, leading companies such as Amazon, Netflix, Spotify, Uber, and eBay rely on microservices to align IT infrastructures with business processes and maintain agility [33]. However, difficulties arise in identifying service

Table 2 Research questions

No.	Research Question	Motivation
RQ1	What are the specifics of anomaly detection in microservices architecture?	Identify the specific aspects related to anomaly detection in microservices.
RQ2	Which approaches can be used for anomaly detection in microservices?	Investigate types of approaches and provide an organized grouping of the various methods by these types.
RQ3	How can the causes of these anomalies be identified?	Identify methods used to identify RCs.
RQ4	What is the performance of the various identified methods?	Indicate the performance of the methods to identify areas of improvement.
RQ5	Which testbeds or datasets can be used in future investigations?	Identify Datasets and platforms to allow future experimental work.

boundaries, managing security, optimizing communication, and ensuring performance at scale [34].

Compared with monolithic systems–where all processes are tightly coupled and even minor changes require redeployment of the entire application [35], microservices offer greater scalability and flexibility. Instead of resizing a whole system, resources can be scaled only where needed, making microservices more suitable for cloud-native deployments [36].

3.2 Architecture of microservices

Microservice-based architecture is an organizational approach to software development in which applications consist of small, autonomous services communicating through well-defined APIs [36]. These services are typically developed and maintained by small teams with significant autonomy.

Although there is no strict standard, Fowler [32] and subsequent work have identified several recurring characteristics. **Service components** ensure that system functionality is decomposed into small units that interact through lightweight interfaces rather than shared libraries, promoting modularity and independence. **Business-oriented organization** means that services are aligned with business domains and supported by cross-functional teams, allowing technical design to directly reflect organizational goals. **Decentralized data management** gives each service control over its own database technology, ensuring that data storage solutions are tailored to service-specific needs. **Technology flexibility** allows teams to adopt different programming languages, frameworks, and storage technologies best suited to their purposes [37]. **Fault tolerance** is a fundamental principle, as services are designed to degrade gracefully and depend on monitoring and recovery mechanisms to maintain stability. Finally, **DevOps alignment**

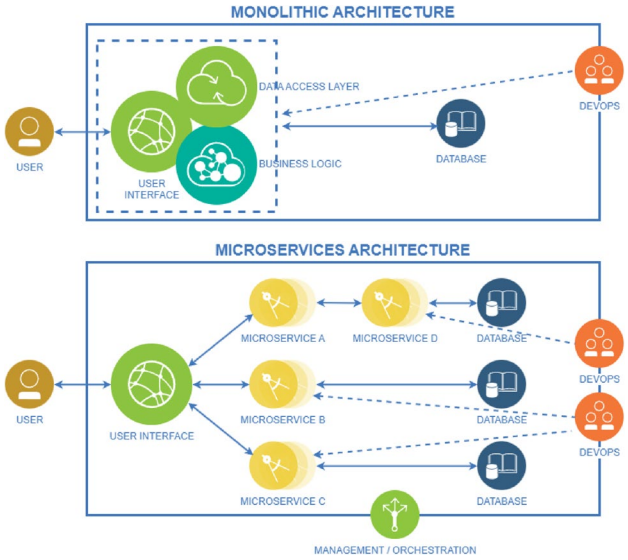

Fig. 1 Microservice vs. Monolithic architectures (adapted from [38] and [39])

enables teams to manage the entire lifecycle of a service–from design and implementation to testing, deployment, and maintenance–within the same domain of responsibility.

Unlike monolithic systems, microservice applications are modular and loosely coupled. This ensures that modifying one service rarely requires changes to others (Figure 1). The single-responsibility principle [6] reinforces this design by grouping together components that evolve for similar reasons while isolating those that change for different ones, thereby enabling independent development and deployment.

3.3 Advantages of microservices

In essence, many benefits arise when microservices are implemented correctly, making full use of distributed systems [6]. **Heterogeneity** allows collaboration between services to be implemented in different ways, since each team can freely choose the programming language, framework, or technology stack best suited to its service. **Resilience** is achieved because microservices are built and tested independently, with clear service boundaries that facilitate the identification and isolation of problems in the event of failures. **Scalability** enables organizations to scale only the services that require additional resources, in contrast to monolithic applications where the entire system must be scaled regardless of which component is under pressure. Finally, **facilitated deployment** is possible because microservices are small, self-contained units that can be modified and deployed individually, avoiding the risks and costs of redeploying an entire monolithic system.

3.4 Challenges in microservices architecture

Despite their advantages, microservices may not be the best option for certain projects. In general, the main drawback lies in the intricacy of any distributed system [40]. The scope and identification of microservices require a deep understanding of the application domain, and defining clear boundaries and contracts between services can be challenging; once established, modifications may demand significant time and effort. As distributed systems, microservices must also address assumptions regarding state, consistency, and network reliability, and because services often span heterogeneous networks and technologies, entirely new failure modes emerge.

Microservices also raise significant operational and maintenance concerns [41]. They require the coordination of numerous services, which increases cost and complexity compared to monolithic systems that only run a single application. The host environment is often intricate, with complex domain hierarchies and layered network architectures. Management typically relies on container orchestration platforms such as Kubernetes, where service call dependencies can become highly entangled. Monitoring further complicates operations, as the system must track multiple metrics across services, containers, and servers, with indicators that fluctuate dynamically. Microservices evolve quickly, with frequent iterations and versioning that make it difficult to maintain stability and ensure timely upkeep in production environments.

3.5 Introduction to anomalies

Anomaly detection is the task of identifying patterns in data that deviate from expected behaviour. Depending on the domain, such patterns are also referred to as outliers, aberrations, exceptions, or discordant observations [22]. Typical applications include intrusion detection in cybersecurity, abnormal condition detection in healthcare, and fault detection in critical infrastructures.

At first glance, anomalies could be defined as any data point outside a "normal" region. In practice, however, several challenges arise:

- Defining a boundary that captures all valid normal values is difficult;
- Many models require labelled data for training and validation, which is costly or unavailable;
- The notion of what constitutes an anomaly varies across domains.

These challenges are amplified in large-scale microservice infrastructures, where services are continuously added, removed, scaled, or migrated, making it difficult to establish stable baselines of normal behaviour.

Three main categories of anomalies are commonly considered [22, 42]:

- **Point Anomalies:** A single data instance significantly deviates from the rest (the most studied type);
- **Contextual Anomalies:** An observation is anomalous only in a given context, but normal in another;
- **Collective Anomalies:** A set of related data points appears abnormal when viewed together, often in sequential or time-series data.

3.6 How to deal with an anomaly

Anomalies may result from human errors, equipment malfunctions, natural variations, fraudulent activities, or system faults. Their treatment depends on the application domain: for example, a typographical error can be corrected and restored to normal, whereas an instrument reading error may simply be discarded [43]. In contrast, anomalies in critical domains such as fraud detection or intrusion monitoring must be detected immediately, stored separately, and used as references for future incidents.

To address these cases, three main approaches to anomaly detection are commonly identified [43]:

- **Type 1 - Unsupervised:** Assumes anomalies differ from normal data and manifest as outliers. Clustering-like methods analyze distributions and flag distant points, but they struggle with dynamic or non-stationary data.
- **Type 2 - Supervised:** Requires labelled normal and abnormal examples. Classifiers trained on such data can perform real-time anomaly detection, provided both classes are well represented and generalization is strong.
- **Type 3 - Semi-supervised:** Learns only the normal class, then flags novel data outside this boundary as anomalies. This approach is suited to evolving systems since it can incrementally adapt to new data.

3.7 Types of anomalies and debugging process in microservices

Outliers are incidences that occur with very low odds of occurrence and are classified into the following categories: (1) an error resulting from a node malfunction, (2) an occurrence triggered by an abrupt or significant shift, (3) a point anomaly that deviates substantially from other data points, (4) a contextual anomaly considered unusual within a specific context, and (5) a collective anomaly, representing a cluster of data that significantly diverges from the remainder of the dataset [45].

Zhou *et al.* [46] collected the types of anomalies based on an industrial survey performed by some engineers with relevant experience in Microservices architecture. In this study, the faults were divided into functional and nonfunctional groups. Functional failures are characterized by errors in applications or incorrect values, whereas non-functional failures result in performance or reliability issues. Cao *et al.* [44] later compiled different anomalies similar to classification trees. We balanced the tree based on their work, emphasizing the security anomalies, as shown in Figure 2.

Owing to the large number of relationships and interactions between different microservices, debugging is much more complex and diverse than debugging a monolithic system. Based on various testimonials [46], the debugging process is divided into seven phases: Initial Understanding (IU), Environment Setup (ES), Failure Reproduction (FR), Failure Identification (FI), Fault Scoping (FS), Fault Localization (FL), and Fault Fixing (FF).

Some phases can be skipped depending on the operator experience and knowledge of previous faults. This process relies on analysing system data, application logs, and monitoring tools. Advanced techniques, such as visual trace analysis, can be used to improve the accuracy of the process.

3.8 Metrics for performance evaluation of anomaly classification

The performance of various methods or algorithms needs to be validated to compare the efficiency and results; therefore, it is vital to understand the metrics used. Precision, Recall, Selectivity, Accuracy, and F1-Score [47] are metrics used by various authors to demonstrate the efficiency of their proposed methods. There are four key concepts behind metrics for the performance evaluation of anomaly classification:

- **True Positives (TP):** Cases where the model accurately detects occurrences as part of the positive class;
- **False Positives (FP):** Cases where the model falsely classifies instances as part of the positive class, even though, in actuality, they are members of the negative class (erroneously marked as positive);
- **True Negatives (TN):** Cases where the model correctly recognizes instances as part of the negative class;
- **False Negatives (FN):** Cases where the model incorrectly identifies instances as part of the negative class when, in reality, they are members of the positive class (wrongly marked as negative).

Precision plays a crucial role, as it assesses how well a model can recognize instances belonging to a particular class in a dataset while minimizing false positives (FP). To calculate precision, we divide the number of TP by the total instances identified by the model as belonging to that class (TP and FP). A method with 100% precision has no false-positive events.

Generally associated with precision is the metric Recall, which measures the accuracy of a model in locating every pertinent instance of a given class in a dataset, resulting in the ratio between these two classifications. A method with 100% recall has no false-negative events.

The F1-score provides the harmonic mean of precision and recall, allowing us to determine the achievement of

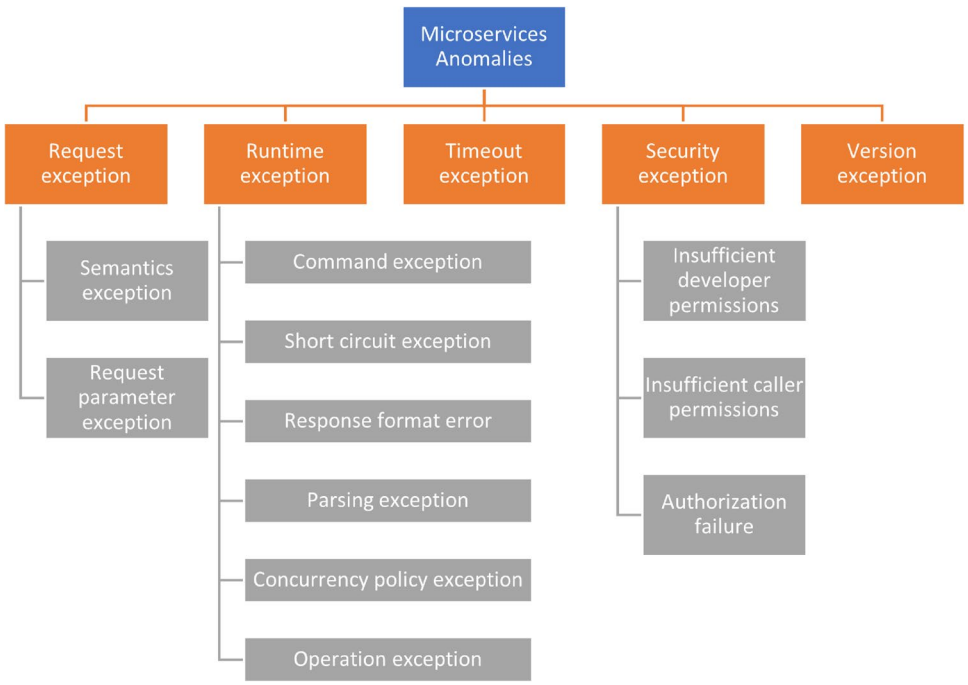

Fig. 2 Microservice anomaly classification tree (adapted from [44])

Table 3 Metrics used to compare methods performance

Metric - Explanation	Equation
Precision - Calculates the percentage of all predicted positives that were accurately identified.	$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$
Recall - Determines the fraction of actual positives correctly identified by the model.	$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$
F1-score - Provides a single metric for the overall performance of the model by striking a balance between precision and recall.	$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
Accuracy - Measures the overall correctness of predictions, combining true positives and true negatives.	$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}$
Selectivity - Quantifies the ability of a model to correctly identify negative instances as negative.	$\text{Selectivity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$
False Positive Rate (FPR) - Represents the proportion of negative instances incorrectly classified as positive.	$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$
False Negative Rate (FNR) - Indicates the proportion of positive instances misclassified as negative.	$\text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}}$
Geometric mean (G-mean) - Measures the overall performance of a model, considering both accuracy and the balance between sensitivity and specificity.	$\text{G-mean} = \frac{\text{TP} \times \text{TN} - \text{FP} \times \text{FN}}{\sqrt{(\text{TP}+\text{FP})(\text{TP}+\text{FN})(\text{TN}+\text{FP})(\text{TN}+\text{FN})}}$
Specificity - Measures the proportion of true negatives (instances correctly classified as negative) out of all negative instances.	$\text{Specificity} = \frac{\text{TN}}{\text{FP} + \text{TN}}$
Negative Predictive Value (NPV) - Measures the proportion of negative test results that are truly negative.	$\text{NPV} = \frac{\text{TN}}{\text{FN} + \text{TN}}$
Area Under the Curve (AUC) - Shows the likelihood that a model ranks a random positive instance higher than a negative one.	$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}^{-1}(x))\, dx$
Mean Reciprocal Rank (MRR) - Used in evaluating search and recommendation systems.	$\text{MRR} = \frac{1}{\lvert Q \rvert} \sum_{i=1}^{\lvert Q \rvert} \frac{1}{\text{rank}_i}$

TP - True Positives, TN - True Negatives, FP - False Positives, FN - False Negatives, TPR - True Positives Rate, FPR - False Positives Rate

Table 4 Keywords used in research process

Keywords	Date Interval
microservice* abnormal*	2012 - 2025
microservices fail*	2012 - 2025
microservice* fault*	2012 - 2025
microservice AND anomalies	2012 - 2025
microservice* AND abnormal*	2012 - 2025
micro-service* abnormal*	2012 - 2025
microservice anomalies	2012 - 2025

better results as a method of relating precision and recall. F1-score is often used in classification tasks where the classes are imbalanced. It provides a balanced evaluation of the performance of a model across all classes. When the F1-score is high, the model performs well by accurately and consistently recognizing instances belonging to the class of interest. In contrast, when the F1-score is low, this indicates that the model has difficulties because it produces a significant number of FP or FN.

Accuracy is calculated by comparing all correct predictions, including both true and false positives, to the total number of predictions made by the model. Although this definition is easy to understand, it may not be the most precise metric. Therefore, it is prudent to complement accuracy using Precision and Recall metrics for a more comprehensive evaluation.

For example, in imbalanced datasets, where the count of instances in each class is unequal, accuracy can be misleading, as it can be high even if the model performs poorly on the minority class.

Some studies refer to the Selectivity metric as a method to validate the performance of the method. This metric is used when the existence of false negatives is a problem, and this metric shows the ability to cover all true negatives. Selectivity is a concept that is frequently employed in classification tasks when the majority class in a dataset represents a negative class. It serves as a gauge of how accurately a model can detect instances that do not belong to the class of focus. A high selectivity score suggests that the model correctly recognizes the negative class, whereas a low selectivity score implies that the model generates a significant number of false positives. Selectivity is calculated as the ratio of true negatives to the total number of negative instances in the dataset.

Accuracy and precision can be more precisely measured using ranking-based metrics such as Precision at Rank 1 (PR@1), Precision at Rank 5 (PR@5), PR@Avg (Average Precision), Accuracy at Rank 1 (AC@1), Accuracy at Rank 5 (AC@5), and Average Accuracy (AC@Avg). These metrics focus on the accuracy and precision of top-ranked results. For instance, AC@1 is particularly relevant when the system must prioritize the accuracy of the first recommendation or prediction. This is similar to PR@1, which

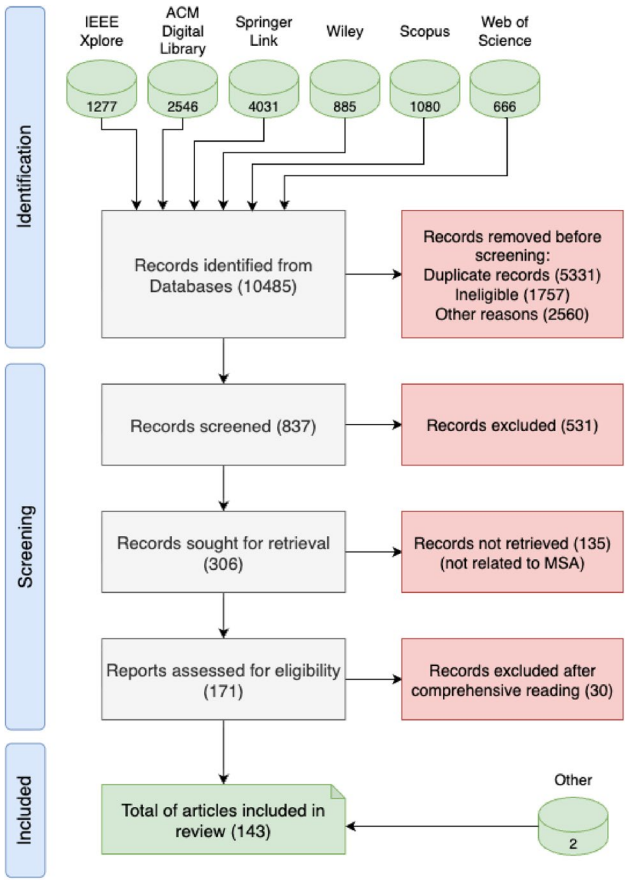

Fig. 3 PRISMA Flow Diagram

focuses on the precision of the top-ranked result–often critical for user satisfaction, system efficiency, and decision-making across various applications. Additional performance metrics, including FPR, FNR, G-mean, and Specificity, are also discussed in the literature [48] [49].

An additional performance metric is the Negative Predictive Value, often referred to as true negative accuracy or inverse precision. This value indicates the proportion of negative cases correctly identified as negative.

Beyond performance evaluation, these metrics also contribute to the quantification of trustworthiness in anomaly detection and root-cause identification systems. Trust can be operationalized through measurable indicators such as detection reliability (stability of Precision, Recall, and F1) [50], diagnostic consistency (variance of root-cause ranking across executions) [51], causal validity (alignment between predicted and actual fault sources) [52], and robustness (performance degradation under noisy or incomplete telemetry) [53]. These measurable properties enable the systematic justification and comparison of trustworthy diagnostic approaches.

4 Anomaly detection in a microservices environment

This section describes the process guiding the specialized literature review, namely the sources of publications, time interval and selection process.

4.1 Literature search strategy

We started by defining the search strategy and methodology and initial analysis of the retrieved results, focusing on the author's affiliation and countries and identifying some related authors and groups of investigators.

4.1.1 Time interval

The search was conducted between December 2021 and May 2025. We included publications after 2012 retrieved from the most significant Digital Libraries in these subjects, namely, "IEEE Xplore", "ACM Digital Library", "Springer Link", "Wiley", "Scopus", and "Web of Science", including only studies written in English.

4.1.2 Keywords

To better identify studies related to the topic, several keywords were used and compiled in Table 4 to allow the replication of this research process.

4.1.3 Results retrieved

In our preliminary research, we identified 63 relevant publications retrieved from the six Digital Libraries. While this served as our initial dataset, our investigation extended further as additional publications were retrieved and reviewed based on alerts created in various Digital Libraries consulted during analysis and writing. Based on these warnings, 80 new publications were included in our review to ensure that our survey was as up-to-date as possible until the final writing phase.

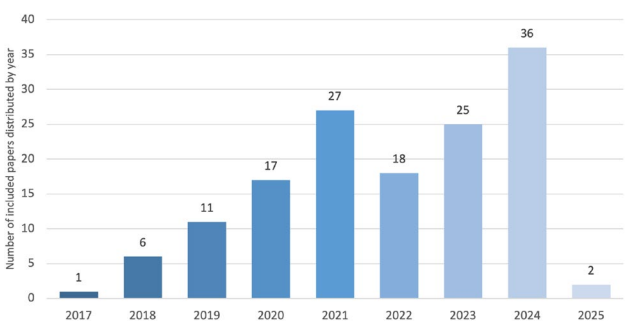

Fig. 4 Studies included by year of publication

The entire process is described in our replication package [54], which provides documentation and source data to ensure full transparency and reproducibility of the research process.

4.1.4 Selection process

The process started with seven initial queries, as described in Figure 3, from which 10485 publications were identified; thus, the process of finding duplicates was initiated.

In this automatic process, the duplication key was the title of the publication, and the results were analysed by the authors by applying the following exclusion criteria:

- Short studies (less than four pages);
- Non-primary studies (surveys, literature reviews, etc);
- Books or Chapters;
- Studies not written in English.

After this initial selection, 9648 publications were discarded, and the screening process was initiated with 837 resulting works. The exclusion criteria were as follows:

- Title not referring to anomaly detection or root-cause identification;
- Studies without full text available.

The remaining 306 articles were examined after excluding 531 articles (abstract, introduction, conclusion, and article structure) using the following exclusion criteria:

- Studies not offering details on anomaly detection or root-cause identification in microservices.

Following this process, all 171 remaining articles were read comprehensively, and the following inclusion criteria were applied to select the studies to be included in our review:

- Studies related only to microservice-based systems;
- Studies with primary emphasis on anomaly detection and/or root-cause identification in microservice-based systems;
- Studies including solutions, methodologies, security mechanisms, and other procedures to handle the detection and identification scope.

After applying the exclusion and inclusion criteria, 143 publications were selected for inclusion in our survey; 141 from the direct application of the PRISMA methodology, plus 2 publications retrieved from other methods (cited on other publications).

Figure 4 presents a noteworthy observation: the distribution of the included studies. Analysing their publication years reveals that 86% were published between 2020 and 2024, highlighting the recent emergence of this topic. Additionally, the scarcity of publications before 2017 further underscores its novelty.

In the literature review, several approaches arose from publications. Most of the methods and algorithms of the authors combine two complementary systems, defining a system capable of identifying an error or fault, followed by another system capable of identifying the reason for that fault by investigating the leading cause of the failure.

This division is followed by the organization of this survey, identifying several approaches and techniques. When analysing the country of the authors, there was an enormous prevalence of authors affiliated with Chinese institutions or organizations, representing 70% of all studies included in this survey. Canada represents 6%, followed by Portugal and the USA at 3%. With an overall rate of 2%, we can identify contributions from Taiwan, Italy, Sweden, Switzerland, and Germany. Another interesting observation is the prevalence of authors from the Northern Hemisphere.

If we analyse the works published in the past 12 months, we observe a significant contribution from Chinese authors, accounting for 87%. This dominance can also be attributed to the large number of authors involved in each publication, often with 11, 12, or even 15 authors.

After the presentation of the investigation process and methodology, let us focus on the various phases of anomaly detection in microservices. We will start by collecting data and information on the system state, followed by identifying abnormal data and the source of the anomalies (Root-Cause).

4.2 Data collection methods

The variety of technologies, vast number of microservices, and rapid modifications in software features and infrastructure make it challenging to identify anomalies in microservices because the system state and organization constantly change. Most studies included in this survey shared this idea.

The discovery of anomalies is the starting point for the root-cause localization process. The first stage starts by collecting data and information from the monitoring system. There are several approaches for collecting data: log, distributed trace, and monitoring-based approaches.

Conventional anomaly diagnosis techniques often rely on thresholds for Key Performance Indicators (KPIs) [41]. System administrators manually define these values based on their knowledge of the system. Owing to the vast quantity of services in a microservice framework and their extensive relationships, system administrators sometimes have difficulty recognizing anomalies and determining their RCs.

The complexity of microservice components, deep interdependencies across services, and frequent system updates increase the risk of failure and the difficulty in identifying issues. When one of the components functions improperly or fails, the impact propagates through the component calls, reducing the overall integrity of the application.

The service response time is the most commonly used method for detecting anomalies in microservices. An immediate rise in response time might be a sign of a problem with the distributed system or the service itself. In anomaly detection, statistical analysis is frequently employed to compute the mean, standard deviation, and median, among other metrics of huge datasets. It is important to gain a comprehensive understanding of the data and conduct fundamental [55] checks for reliability and completeness.

Latency is typically used to explore UI-related services, whereas computation-intensive services (such as Hadoop or Spark computations) can be diagnosed using throughput or CPU metrics. While service calling requirements are constantly changing, the functionalities of microservices are relatively consistent [56] [57].

Data-driven judgments have the drawback of potentially being impacted by the calibre of the measured data, especially data abnormalities. Real-world data are often imprecise and partial and contain incorrect values. By failing to discover and filter out such data abnormalities, the results of data-driven choices may be significantly distorted [58].

4.2.1 Log based

The examination of log files is one of the most typical approaches for anomaly detection and fault diagnosis in microservice-based systems [41]. Logs, generated by software source code and logging statements, are semi-structured text files that often represent the only source of runtime information. Their importance has grown as modern software systems scale, but the increasing volume and heterogeneity of logs make their analysis a significant challenge [59, 60]. Traditional anomaly detection methodologies for distributed and cloud-based systems continue to rely heavily on log analysis [61, 62].

Several methods exemplify different strategies for leveraging logs. Some works focus on the lifecycle of cloud applications, where tools such as LogDC analyze Kubernetes logs end-to-end [63], while others optimize for large-scale data by clustering RPC chains and applying spatial-temporal models [64]. Kernel-level monitoring has also been explored, with approaches such as DyCause capturing API interactions, request paths, response codes, and latency measures to diagnose service performance issues [65]. For

large-scale platforms, anomaly detectors integrated directly with existing log infrastructures emphasize scalability and elasticity, adapting to dynamic and serverless architectures [20]. Multi-stage pipelines have also been proposed, combining feature extraction, normalization, and classification of Kubernetes container logs [66].

Deep learning models further extend log-based analysis. Techniques such as DeepLog [67] have been adapted to compute anomaly scores in microservice systems [68], while causality-based models order log events using *happens-before* relations to strengthen root-cause identification [69]. More recently, hybrid frameworks combine offline training with selective online monitoring, reducing data volume while preserving accuracy [70].

Despite their usefulness, log-based methods also present limitations. Not all abnormal actions are recorded in logs, requiring operation and maintenance staff to combine domain expertise with automated mining [41]. Moreover, logs are not sufficient to capture anomalies in service quality performance metrics or to fully track anomaly propagation in distributed systems [63]. Thus, while log-based approaches remain a cornerstone of anomaly detection, they are most effective when complemented by additional data sources and integrated methods for root-cause analysis.

4.2.2 Distributed tracing based

Tracing, while conceptually related to logging, focuses on capturing the program flow as requests propagate across modules and service boundaries [71]. Distributed tracing has become essential for benchmarking, diagnosing, and debugging microservice systems, as it records execution pathways and reveals inter-service dependencies. These traces, however, are produced at massive scale, making them expensive to store and process [72]. Due to the complexity and dynamism of microservices, static analysis and logs alone are often insufficient, and trace analysis is required to understand system behavior and localize anomalies [73].

Several works propose methods for managing trace data and extracting useful features. Early research formalized trace structure into spans, parent–child relationships, and identifiers, enabling hierarchical analysis of user requests [71, 74]. However, high storage and runtime costs demand sampling strategies, where random head-based or selective tail-based sampling reduce overheads while preserving representative traces [75]. Approaches like ES-APM [76] efficiently record request paths and latencies, while others emphasize capturing both local and remote calls, which complicates analysis compared to non-distributed systems [69]. At scale, systems like eBay generate billions of traces daily, requiring near real-time aggregation and on-demand access [73].

To process such data, diverse anomaly detection strategies have been explored. Some rely on graph-based or service dependency models, e.g., GMAT and GMTA [73, 77], which construct service dependency graphs (SDG) or abstract traces into business flows for visualization and diagnosis. Others use deep learning, such as TraceAnomaly [78], which learns typical trace patterns offline and scores anomalies online, or methods leveraging VAEs for categorical grouping of traces [79]. Ranking-based methods like TraceRCA [74], Microrank [80], T-Rank [81], and TraceRank [82] combine anomaly detection modules with ranking techniques to prioritize fault localization. Other techniques represent traces as graphs (e.g., PUTraceAD [83]) or causal models, while approaches such as TProf [84] balance coarse-grained profiling for scalability with fine-grained tracing for detailed diagnosis.

Noise and variability are central challenges, as systems incorporate caching, load balancing, and dynamic service instances. Trace analysis must therefore recognize unseen but valid execution sequences as normal [85]. Some works address this by parallelizing collection and processing pipelines [81], or by simplifying feature sets to reduce overhead [86]. Hybrid methods combine multiple data sources: for instance, Zuo *et al.* [87] fuse log and trace data for improved anomaly detection. Security-oriented approaches also emerge, where deviations in system call sequences are identified from trace data to detect attacks [88].

Open standards like OpenTracing underpin many of these systems [71, 89, 90], enabling language-agnostic instrumentation and widespread adoption in industry (e.g., Apple, Uber). These frameworks facilitate downstream analysis, such as extracting metrics from traces and feeding them into time-series databases for anomaly detection [71]. Overall, distributed tracing provides a granular, end-to-end perspective of microservice execution, but its effectiveness depends on balancing the granularity and volume of trace data with the scalability of analysis pipelines.

4.2.3 Monitoring based (metric based)

Monitoring-based approaches acquire data from system and application metrics to detect anomalies in microservice environments. These methods are widely adopted since they can be integrated into runtime monitoring infrastructures without requiring access to source code. Early work demonstrated federated monitoring systems for IoT platforms [91], while subsequent research focused on performance indicators such as response time, latency, and throughput, which often signal service degradation [55, 69, 92]. However, noise and variability across services complicate the modeling of response times, making it necessary to correlate multiple metrics and enrich monitoring data with service context.

Several contributions illustrate how diverse metrics are collected and analyzed. In addition to latency and throughput, many systems monitor CPU and memory utilization [93, 94], network traffic [95], cache metrics, and I/O operations [14, 44, 96]. Tools such as Prometheus and service meshes like Istio are frequently employed to collect these indicators across microservices, containers, and nodes [97]. Application-independent approaches such as MicroRCA [98] and RAD [99] further extend monitoring to application and VM levels, while DockerAnalyzer [100] emphasizes the container and VM abstraction layers. The heartbeat mechanism has also been explored as an active probing strategy, sending requests to services and raising alerts when responses are delayed or missing [101].

Beyond raw metric collection, many systems apply statistical and learning techniques to identify anomalies. Causality-based models derive service impact graphs from latency and throughput correlations [92], while change point detection methods capture early signs of performance degradation in KPI time series [62]. More advanced frameworks apply feature selection and scalability optimization, as in DICE Monitoring [61], or address tail latency issues specific to microservice workloads [102]. Recent studies also highlight the importance of correlating system-level and application-level metrics [103, 104], enabling earlier anomaly identification and more accurate root-cause localization.

Scalability and generalization remain central challenges. Semi-supervised learning approaches aim to train models that generalize across servers and container instances without requiring per-instance training [105]. Global frameworks such as AutoMAP [96], DLA [106], and recent graph-based models [107–109] integrate multiple layers of metrics into unified representations for robust detection and diagnosis. Overall, metric-based monitoring provides a scalable and effective foundation for anomaly detection, though its success depends on selecting the right combination of metrics and correlating them across system layers.

4.3 Methods to identify anomalies in microservices

During the literature review process, various methods were explored to analyze the collected system data, detect abnormal values, identify anomalies, and indicate their presence. After data collection, machine learning, trace-based, and statistical methods were applied to process the information and detect anomalies.

4.3.1 Unsupervised learning

Unsupervised learning methods are widely applied to anomaly detection in microservices, as they do not require labeled data and can exploit the fact that normal cases are far more common than anomalies [85, 88]. Early work used clustering approaches, such as K-means and Expectation Maximization in LogDC [63], grid-based clustering combined with K-means for monitoring data [91], and clustering applied to distributed traces [82]. Other approaches addressed trace variability, as in TraceRCA [74], which infers abnormal invocations instead of relying on fixed-length vectors.

Clustering remains central to several frameworks. BIRCH clustering has been integrated into MicroDiag [97], MicroRCA [98], and AAMR [103], where it is applied to response times and service dependencies to identify anomalies and propagation paths. Informer [64] extended the analysis of large log datasets with Graph Convolution Networks, while ServiceRank [92] relied on causality graphs without labeled data. Graph-based approaches are increasingly relevant: PUTraceAD [83] framed anomaly detection as a positive–unlabeled problem, while TraceGra [11] combined VGAE and LSTM-AE to capture both structural and temporal features. Similar hybrid deep models have been applied to traces, such as TraceAnomaly [78] and TraceSieve [76], which leverage variational autoencoders and adversarial training.

Deep neural networks are also widely adopted. RNNs and LSTMs have been used to capture temporal patterns in time-series data [110], extended with autoencoders for sequence reconstruction and anomaly scoring [111, 112]. Autoencoder-based methods exploit reconstruction errors to flag deviations, while adversarial and contrastive learning frameworks, such as UAC-AD [113], improve robustness against hard-to-classify samples. Other advanced methods include Deep SVDD for hypersphere boundary learning [108], CNN–LSTM agents in distributed settings like MAAD [114], and hybrid frameworks like COAD [115] that integrate real-time feature selection.

Recent contributions also explore human-in-the-loop strategies. Duan *et al.* proposed ACLog [116] and AFA-Log [117], combining unsupervised models with active learning to refine anomaly detection using human feedback, particularly on uncertain or noisy sequences. Other lightweight approaches include SVMs for API traffic [95], randomized matrix sketching for streaming data [118], and regression-based models such as Syslrn [70] that reduce data requirements through offline feature selection. Overall, unsupervised methods span clustering, graph-based learning, autoencoders, and hybrid frameworks, offering scalable solutions to anomaly detection in dynamic microservice environments.

4.3.2 Supervised learning

Supervised learning techniques rely on labeled datasets to classify both normal and anomalous behavior, enabling

models to learn general patterns and detect deviations [88]. These methods address limitations of unsupervised approaches, such as the inability to distinguish between noise and anomalies. For example, Luo *et al.* [119] showed that exposure to noisy data improves model robustness, allowing distinction between minor anomalies, noise, and significant faults.

Semi-supervised learning extends this paradigm by combining labeled and unlabeled data, thereby adapting models to fluctuations in dynamic microservice environments [55]. Examples include BiLSTM and BERT models for log analysis with attention mechanisms [120], hierarchical HMMs trained on CPU, memory, and network metrics [106], and VAE-based detection with high precision [105]. Sliding-window and temporal partitioning techniques further reduce false positives and capture time-dependent anomalies [55]. Other semi-supervised contributions include GRU Autoencoders retrained online for adaptability [20], VAEs applied to multimodal data [105], and frameworks such as AutoLog [60] that exploit normative logs to train baseline autoencoders.

Deep learning is frequently applied in supervised and semi-supervised contexts. LSTM-based models predict task templates or multimodal dependencies [87, 121], while combinations of LSTM and GRU with GNNs capture spatiotemporal dependencies [122]. GNNs also underpin frameworks such as TDAD [123] and TraceGSAD [124], which model spans and traces as graphs to identify abnormal execution paths. Tree-based supervised methods also appear, such as TraceArk [125], which integrates XGBoost with engineer feedback for interpretable anomaly detection. Gradient-based optimization has been applied to DNN classifiers trained on normal and injected-fault traces [90], while adaptive pattern learning allows models to evolve as new data arrives [104].

Classical supervised techniques also remain relevant. Conditional Random Fields have been applied to sequence labeling of abnormal events [44], Gaussian Mixture Models (GMM) were used for clustering anomalies in GRALAF [126], and SVMs for traffic classification at the API level [95]. Hybrid strategies combine supervised classification with graph models or spatiotemporal analysis, demonstrating the diversity of supervised approaches. Overall, supervised and semi-supervised learning offer precise anomaly detection and resilience to noise, but require high-quality labeled data and mechanisms to adapt to the evolving behavior of microservice systems.

4.3.3 Reinforcement learning

Reinforcement learning (RL) differs from supervised and unsupervised methods by relying on interaction with the environment through feedback and rewards: correct behaviors yield positive rewards, while incorrect ones yield negative rewards, enabling models to iteratively improve their decision-making [127].

In microservice anomaly detection, RL has been explored to capture both local and global system behavior. Belhadi *et al.* [128] proposed a multi-agent framework in which each agent detects local anomalies within a microservice using an action–reward strategy, while inter-agent communication enables the aggregation of local insights into global anomaly detection across distributed and heterogeneous architectures. This demonstrates the potential of RL for adaptive, decentralized anomaly detection in dynamic microservice environments.

4.3.4 Trace Comparison

Trace comparison methods analyze structural and temporal similarities between execution traces to identify anomalies and localize root causes. Early approaches such as MonitorRank [129] and FacGraph [130] introduced graph-based ranking to prioritize likely root causes, although these techniques required heavy computation. More recent methods, such as MicroRank [80], combine anomaly detection with PageRank-based scoring, building call and operation–trace graphs to highlight abnormal traces.

Graph databases and causal modeling have been widely employed to capture dependencies in trace data. For example, Neo4j supports graph queries over invocation paths [73], while Neves *et al.* [69] used directed acyclic causal graphs from application logs to analyze runtime behavior. Similarly, Gu *et al.* [131] applied causal graphs over temporal windows to track propagation from anomalies to their root causes. Workflow comparison has also been adopted, with Wang *et al.* [89] and Meng [14] identifying deviations from baseline or tree-matched execution traces.

Recent frameworks emphasize both structural and temporal aspects of traces. Sieve [72] applies resilient random cut forests to sample traces that are either structurally or temporally distinct, improving anomaly detection coverage. Similarly, tprof [84] organizes traces into hierarchical groups based on request types, span structures, and parent–child relationships, enabling aggregate trace representations for scalable analysis. Collectively, these methods demonstrate how comparing trace similarity–whether through graphs, workflows, or structural clustering–enhances anomaly detection and root-cause analysis in microservice systems.

4.3.5 Statistical analysis

Statistical analysis methods form some of the earliest and most widely used approaches for anomaly detection in microservices. These techniques rely on modeling

historical system behavior and identifying deviations in resource utilization or performance metrics. For example, Ravichandiran *et al.* [93] applied univariate autoregressive models over CPU and memory usage, flagging anomalies when observed values deviated from predicted ones. Similar ideas have been extended to service-level metrics such as latency and throughput, where increases in latency combined with reduced throughput signal potential anomalies [56, 130].

Outlier detection using statistical thresholds is a recurring theme. MicroHECL [7], MicroNet [132], and DiagFusion [107] apply the 3-sigma rule to detect extreme deviations in traces, metrics, and invocation latencies. Fourati *et al.* [100] similarly identify container-level anomalies with modified Z-scores based on CPU and memory usage, while elapsed-time analysis has been used as a lightweight trace-based indicator [71]. Version-aware monitoring, as in MS-Version [133], detects anomalies during microservice upgrades by statistically comparing performance across different versions.

Causality-based statistical methods build more sophisticated models of dependencies. DyCause [65, 134] applies Granger causality and Extreme Value Theory to capture latency fluctuations and extreme event structures across APIs. RAD [99] introduces the concept of a "twin workload," comparing real-time response times with baseline predictions derived from queuing network models to detect significant deviations. Related work such as MFRL-CA [62] builds Microservice Fault Propagation Graphs (MFPG) to model and trace fault origins. These approaches highlight how statistical causality analysis can enhance anomaly localization beyond simple thresholding.

More recent work explores hybrid methods that integrate statistical models with unsupervised or signal processing techniques. Sun *et al.* [135] combined clustering of trace logs with similarity measures (e.g., Levenshtein distance, cosine similarity) to detect statistical deviations from normal behavior. Detective-Dee [136] introduced a compressed sensing framework to reconstruct signals from sampled data and identify anomalies in real-time, further optimized with lookup tables and adaptive thresholding. Together, these approaches show the evolution of statistical analysis–from simple autoregression and sigma rules to causality, hybrid clustering, and compressed sensing–towards more adaptive and efficient anomaly detection frameworks for microservices.

All previous methods are compiled in Table 5 and Table 6, where the type of data collection method, anomaly detection method, and type of failures are identified.

4.4 Types of anomalies detected

4.4.1 Workflow anomalies

Workflow anomalies manifest in various forms, each affecting the operations of the system in distinct ways. One such form is the Reliability Anomaly, where an abnormal increase in error counts (EC) signals an issue with service call failures. These anomalies are often triggered by exceptions resulting from coding flaws or environmental shortcomings, such as server or network outages [7].

Another notable anomaly occurs when the system receives two requests containing conflicting information that change the status of a request in contradictory ways. This situation disrupts the normal flow of operations, clearly indicating a workflow anomaly.

Additionally, Traffic Anomalies are observed when there is an abnormal spike or drop in the number of requests per second. A sudden rise in traffic might overwhelm the services and lead to failures, whereas a significant drop may indicate that many requests are not reaching the services. Such traffic irregularities can stem from improper traffic configurations (eg, limitations set in Nginx), DDoS attacks, or unplanned stress tests [7].

The work of Zhou *et al.* [137] emphasizes configuration errors, asynchronous interaction errors, and multi-instance errors as key faults affecting microservice interactions and runtime environments. Similarly, Ma *et al.* [133] highlight that the release of a new version in a continuously evolving system can introduce anomalies – a situation often referred to as Dependency Hell. In systems with tightly coupled dependencies, such upgrades may necessitate a complete redesign of each dependent service.

Furthermore, some authors [124] categorize workflow anomalies into two distinct types:

1. **Sequence Anomalies:** These involve deviations from the normal execution path of a trace, such as changes in the logic of request handling.
2. **Structural Anomalies:** These occur when service instances are either not invoked or are invoked incorrectly, often as a result of service failures.

Each type of anomaly requires careful analysis and targeted mitigation strategies to ensure the stability and reliability of microservice based systems.

4.4.2 Performance anomalies

Performance anomalies are frequently a result of underlying performance issues, system or component failures, or even security breaches.

Table 5 Works on Anomaly Detection in Microservices

Works	Year	Data Collection Method[a]	Anomaly Detection Method[b]	Fault Type[c]	Performance (%)[d]
Ma *et al.* [77]	2018	T	n.d	-	n.a
Pahl and Aubet [91]	2018	M	UL	P/S	FPR: 0.2, A: 99
Ravichandiran *et al.* [93]	2018	M	ST	S	n.a
Zang *et al.* [101]	2018	M	ST	P	n.a
Cao *et al.* [44]	2019	M	SL	P	n.a
Chen *et al.* [64]	2019	L	UL	S	n.a
Drâgan *et al.* [61]	2019	M	UL	P	n.a
Las-Casas *et al.* [75]	2019	T	TR	n.d	n.a
Ma *et al.* [133]	2019	M	ST	W	n.a
Tien *et al.* [66]	2019	L	SL	S	A: 96, F1: 98.1
Bogatinovski *et al.* [85]	2020	T	UL	P	n.a
Chen *et al.* [118]	2020	T	UL	P	n.a
Guo *et al.* [73]	2020	T	TR	n.d	n.a
Luo *et al.* [119]	2020	T	SL	n.d	n.a
Mukherjee *et al.* [99]	2020	M	ST	P	n.a
Zuo *et al.* [87]	2020	L/T	SL	P	n.a
Baye *et al.* [95]	2021	M	UL	S	FPR: 7.3, F1: 96.4
Belhadi *et al.* [128]	2021	n.d	RL	n.d	n.a
Bento *et al.* [71]	2021	T	UL	n.d	n.a
Chen *et al.* [90]	2021	T	SL	W/P	A: 91.5, R: 89, F1: 88.6
Ghorbani *et al.* [111]	2021	M	UL	S	FPR: 0.01, P: +9, A: +14
Huang and Zhu [84]	2021	T	TR	W	n.a
Huang *et al.* [72]	2021	T	TR	P	n.a
Islam *et al.* [20]	2021	L	SL	P	n.a
Jacob *et al.* [88]	2021	T	TR	S	n.a
Meng *et al.* [14]	2021	T	TR	W/P	P: 81–97, R: 75–99, F1: 77.9–98.9
Neves *et al.* [69]	2021	L/T	TR	W/P	n.a
Afshinpour *et al.* [138]	2022	M	n.d	P	A: 71–90
Cao *et al.* [86]	2022	T	ST	S	A: 99.2, F1: 98.2
Catillo *et al.* [60]	2022	n.d	n.d	-	R: 96–99, A: 93–98
Chen *et al.* [104]	2022	M	SL	P	F1: 82.5
Chen *et al.* [121]	2022	L/M	UL	P	P: 71.69, R: 99.53
Huang *et al.* [105]	2022	M	SL	P	P: 89, R: 85, F1: 87
Kohyarnejadfard *et al.* [139]	2022	T	UL	n.d	P: 97.7, R: 97.6, F1: 97.5
Li *et al.* [140]	2022	n.d	UL	P	P: 88, R: 86, F1: 87
Sanvito *et al.* [70]	2022	L	UL	W	R: 98
Silva *et al.* [110]	2022	M	UL	P	n.a
Yang *et al.* [122]	2022	M	SL	P	R: +7.7–25.5.7.5, F1: +2.9–7.6.9.6
Zhang *et al.* [141]	2022	L/T	UL	W	P: 93, R: 97.8, F1: 95.4
Zhang *et al.* [83]	2022	T	UL	P	P: 90.5, R: 98.7, F1: 94.4
Chen *et al.* [11]	2023	T	UL	W/P	P: 97.6, R: 93, F1: 95
Duan *et al.* [116]	2023	L	UL	P	F1: 82,52 - 92.68
Duan *et al.* [117]	2023	L	UL	P	F1: + 6.61
He *et al.* [142]	2023	M	UL	P	A: 94.5–96.3.5.3
Huang *et al.* [109]	2023	L/T/M	UL/SL	P	F1: 95.7 & 96.5
Li *et al.* [143]	2023	L	UL	P	F1: 60.87 & 92.66
Raeiszadeh *et al.* [123]	2023	T	UL	P	P: 87.1, R: 93.0, F1: 89,8
Ren *et al.* [108]	2023	L/T/M	UL	P	P: 89.0, R: 97.0, F1: 88.0
Traini and Cortellessa [144]	2023	T	n.d	P	P: 98, R: 99
Xie *et al.* [145]	2023	T	UL	W/P	F1: 92.3–95.4.3.4
Zaojian *et al.* [120]	2023	L	UL/SL	P	n.a
Zeng *et al.* [125]	2023	T	SL	P	AUC: 99.94

Table 5 (continued)

Works	Year	Data Collection Method[a]	Anomaly Detection Method[b]	Fault Type[c]	Performance (%)[d]
Zhou *et al.* [115]	2023	M	UL	P	F1: +5.67
Ding *et al.* [124]	2024	T	UL/SL	W/P	P: 96.8, R: 94.1, F1: 95.4
Dodda *et al.* [146]	2024	M	UL/SL	P	A: 95.23 - 99.81
He *et al.* [147]	2024	T	UL	P	F1: 90.0 & 91.0
Khudyakov and Yakhyaeva [148]	2024	L/T	UL	P	P: 98.0, R: 98.0
Liu *et al.* [149]	2024	L/M	UL	P	P: 82.7, R: 90.2, F1: 86.2
Liu *et al.* [113]	2024	L/T	UL	P	F1: 92.3
Shahini and Momeni [112]	2024	T	UL	P	P: 73.10, R: 79.9, F1: 76.4
Tan and Li [114]	2024	L/T	UL	P	P: 95.80, R: 99.6
Zeng *et al.* [150]	2024	T	UL	W	A: 97.6, P: 99.80, R: 95.4, F1: 97.5
Zhu *et al.* [151]	2024	M	UL	P	P: 72.29, R: 90.13, F1: 80.23
Osswald *et al.* [152]	2025	L/T	SL	P/S	A: 99.99, P: 99.99, R: 99.99, F1: 99.99
Raeiszadeh *et al.* [153]	2025	L/T	UL	P/W	P: 94.0, R: 98.7, F1: 96,3

[a]Data Collection Method (viz., L - Log Based, M - Monitoring Based, T - Trace Based)

[b]Anomaly Detection Method (viz., UL - Unsupervised Learning, SL - Supervised Learning, RL - Reinforcement Learning, TR- Trace Comparison, ST - Statistical Analysis, n.d - Not described)

[c]Root-Cause Identification Method (viz., ML - Machine Learning Methods, G - Graph Methods, S - Statistical Methods, n.d -Not described)

[d]Type of Anomalies identified (viz., W - Workflow anomalies, P - Performance anomalies, S - Security anomalies)

[e]Performance of the Algorithm (viz., PFR - False Positive Rate, A - Accuracy, P - Precision, R - Recall, F1 - F1-score, S - Specificity, AC@1 - Top-1 Accuracy, AC@3 - Top-3 Accuracy, AC@5 - Top-5 Accuracy, AC@Avg - Average Accuracy, PR@1 - Top-1 Precision, PR@3 - Top-3 Precision, PR@5 - Top-5 Precision, PR@AVG - Average Precision, MRR - Mean Reciprocal Rank, MicroF1 - Micro F1-score, MacroF1 - Macro F1-score, n.a - Not available)

These anomalies leave behind traces in monitored data, including logs, metrics, or distributed traces [85]. Such traces are essential for identifying and diagnosing the root causes of performance degradation within the system.

One of the key indicators of a performance anomaly is an abnormal increase in Reaction Time (RT), which directly suggests that the system is underperforming. This degradation may be attributed to problematic implementations or incorrect environmental configurations, such as suboptimal CPU or memory settings in containers and virtual machines [7].

Performance anomalies can also stem from inefficient resource management or unexpected spikes in workload, leading to system bottlenecks. For instance, an increase in concurrent requests or resource-intensive processes can overwhelm the system, resulting in delayed responses and reduced throughput. In distributed systems, these issues may propagate, causing cascading failures that impact overall service reliability.

Moreover, monitoring tools play a crucial role in detecting these anomalies. By analyzing trends and patterns in system metrics over time, engineers can identify abnormal behaviour before it escalates into critical failures. Techniques such as threshold-based alerting and anomaly detection algorithms are commonly employed to monitor reaction times and other performance indicators, enabling proactive maintenance and timely remediation.

4.4.3 Security anomalies

Chen *et al.* [64] focus their anomaly detection on communication between services (RPCs), trying to automatically identify irregular RPC traffic detection in a simple and practical framework. Malicious users can abuse public APIs or even when some of the containers are compromised, and there could have been unusual changes in RPC traffic.

KubAnomaly, presented by Tien *et al.* [66], offers security monitoring tools for detecting anomalies on the Kubernetes orchestration platform and uses neural network techniques to build classification models that demonstrate the capacity to identify anomalous activity, including expo assaults, web service attacks, and well-known vulnerabilities. SQL Injection, Command Injection, Zap attack, and DDoS are security anomalies detected by the algorithm. In addition, Pahl and Aubet [91] showed that their algorithm could identify DDoS attacks in a federated infrastructure. A more straightforward approach was presented by Ravichandiran *et al.* [93], who compared historical data to actual data and identified diverging values as anomalies caused by the DDoS.

Table 6 Works on Anomaly Detection and Root-cause Identification in MSA

Works	Year	Data Collection Method[a]	Anomaly Detection Method[b]	RCI Method[c]	Fault Type[d]	Performance (%)[e]
Xu *et al.* [63]	2017	L	UL	n.d	n.d	P: 91, R: 92
Lin *et al.* [130]	2018	L	ST	G	P	n.a
Fourati *et al.* [100]	2019	M	ST	ML	P	n.a
Ma *et al.* [56]	2019	M	ST	G	P	n.a
Samir and Pahl [106]	2019	M	SL	ML	P	A: 97
Shan *et al.* [102]	2019	M	UL	G	P	n.a
Zhou *et al.* [137]	2019	T	n.d	ML	W	FPR: 0.9, P: 99.8, R: 98, A: 93, F1: 99
Jin *et al.* [94]	2020	M	SL	ML	P	P: >90
Liu *et al.* [78]	2020	T	UL	G	n.d	P: 98, R: 97, F1: 97
Ma *et al.* [57]	2020	M	ST	G	P	n.a
Ma *et al.* [96]	2020	M	n.d	ML	n.d	A: >90
Wang *et al.* [68]	2020	L	ST	ML	n.d	n.a
Wang *et al.* [89]	2020	T	G	G	W	n.a
Wu *et al.* [98]	2020	M	G	G	P	P: 97
Bento *et al.* [154]	2021	L/T/M	n.d	n.d	P	R: 77, S: 89
Cai *et al.* [79]	2021	T	ML	n.d	-	A: 97
Chen *et al.* [62]	2021	M	TR	G	P	A: +12.75
Hou *et al.* [155]	2021	L/T/M	n.d	n.d	n.d	R: 84.8
Li *et al.* [55]	2021	M	SL	S	W/P	A: 99
Li *et al.* [74]	2021	T	UL	S	S	A: +44.8
Liu *et al.* [7]	2021	M	ST/G	ML/G	W/P	n.a
Ma *et al.* [92]	2021	M	UL	ML	P	n.a
Pan *et al.* [65]	2021	L	ST	G	P	n.a
Wu *et al.* [97]	2021	M	G	G	P	P: 97
Ye *et al.* [81]	2021	T	ST	S	n.d	P: 82, R: 93
Yu *et al.* [80]	2021	T	TR	S	P	R: +6 to 22
Yu *et al.* [82]	2021	T	UL	G	P	P: 90, R: 86, F1: 88
Zhang *et al.* [103]	2021	M	UL	G	P	A: 94
Li *et al.* [156]	2022	T	n.d	ML	P	n.a
Li *et al.* [157]	2022	M	UL	G	P	n.a
Yang *et al.* [158]	2022	T	TR	S	P	P: 87
Yang *et al.* [159]	2022	M	UL	ML/G	P	P: 90.4
Zhang *et al.* [160]	2022	L/M	UL	S	W	P: 86, R: 83
Gu *et al.* [131]	2023	L/T/M	TR	S	W/P	AC@1: 90, AC@Avg: 90
Jiang *et al.* [161]	2023	L	n.d	G	P	n.a
Kalinagac *et al.* [162]	2023	M	UL	S	P	n.a
Kalinagac *et al.* [126]	2023	M	UL	ML	W/P	A: 95.9, R: 96.3, FPR: 4.5
Li *et al.* [163]	2023	L	UL	G	P	P: 94, R: 95.8, F1: 89
Pan *et al.* [134]	2023	L	ST	S	P	PR@1: 66.67, PR@AVG: 93.33
Sun *et al.* [135]	2023	L/T	S/UL	G	P	AC@1: 50, AC@Avg: 72.22
Xin *et al.* [164]	2023	M	n.d	G	P	AC@Avg: 58 - 66
Zhang *et al.* [76]	2023	T	UL	G	P/W	F1: 97 & 92.5
Zhang *et al.* [107]	2023	L/T/M	UL	ML	P	P: 86.0, R: 82.9, F1: 83.9
Borse *et al.* [165]	2024	L/T/M	UL	ML	P/W/S	A: 99.4
Chen *et al.* [166]	2024	L/T	UL	ML/G	P/W	AC@3: 95.9 - 100
Chen *et al.* [167]	2024	M	SL	ML/G	P	P@1: 99.0, P@2: 100
Cheng *et al.* [168]	2024	T/M	UL	ML/G	P/W/S	AC@1: 90.5 - 93.0
Enz *et al.* [169]	2024	M	UL	ML	P	A: 97.7, P: 94.4
Gao *et al.* [170]	2024	M	ST	G/S	P	PR@5: 48.5
Han *et al.* [171]	2024	L/T/M	SL	ML/G	P/W/S	AC@5: 94.09
Han *et al.* [172]	2024	L/T/M	SL	ML/G/S	P/W/S	AC@5: 94.54

Table 6 (continued)

Works	Year	Data Collection Method[a]	Anomaly Detection Method[b]	RCI Method[c]	Fault Type[d]	Performance (%)[e]
Huang *et al.* [173]	2024	M	SL	ML/S	P	MacroF1: 82.18; MicroF1: 97.11
Huang *et al.* [174]	2024	T/M	SL	G/S	P/W	F1: 93.0, AC@1: 42.0, AC@3: 61.0, AC@5: 80.0
Li *et al.* [175]	2024	T/M	UL	ML/G	P/W	AC@1: 69.54, AC@5: 94.09
Man *et al.* [136]	2024	L/M	ST	S	P	n.a
Pham *et al.* [176]	2024	M	UL	G/S/ML	P	n.a
Ren *et al.* [177]	2024	L/T/M	SL	S/ML	P/W	AC@1: 91.32 - 93.1
Sun *et al.* [178]	2024	L/T/M	UL	ML	P/W	F1: 94.2
Tao *et al.* [179]	2024	M	ST	G	P	MRR: 70.0 - 78.0
Wang *et al.* [180]	2024	T	UL	G	P/W	F1: 91.9 - 93.2
Wang *et al.* [181]	2024	L/T	UL	ML	P/W/S	PR@1: 73.8, F1: 89.3
Wang *et al.* [182]	2024	M	ST	S	P	P: 91.0, F1: 90.0
Wu *et al.* [183]	2024	M	ST	G/S	P/W	AC@1: 89.8, AC@Avg: 97.1
Xu *et al.* [184]	2024	M	UL	G	P	A: +10
Yang *et al.* [132]	2024	T	ST	G/S	P	P: 87,0
Yao *et al.* [185]	2024	T	UL	G/S	P	AC@1: 66.1, AC@3: 86.4, AC@5: 88.1
Zhang *et al.* [186]	2024	M	ST	G/ML	P	PR@1: 86.7 - 96.7
Zhang *et al.* [187]	2024	T	ST	G/S	P	MRR: 55.4 - 70.6

[a]Data Collection Method (viz., L - Log Based, M - Monitoring Based, T - Trace Based)

[b]Anomaly Detection Method (viz., UL - Unsupervised Learning, SL - Supervised Learning, RL - Reinforcement Learning, TR - Trace Comparison, ST - Statistical Analysis, n.d - Not described)

[c]Root-Cause Identification Method (viz., ML - Machine Learning Methods, G - Graph Methods, S - Statistical Methods, n.d - Not described)

[d]Type of Anomalies identified (viz., W - Workflow anomalies, P - Performance anomalies, S - Security anomalies)

[e]Performance of the Algorithm (viz., PFR - False Positive Rate, A - Accuracy, P - Precision, R - Recall, F1 - F1-score, S - Specificity, AC@1 - Top-1 Accuracy, AC@3 - Top-3 Accuracy, AC@5 - Top-5 Accuracy, AC@Avg - Average Accuracy, PR@1 - Top-1 Precision, PR@3 - Top-3 Precision, PR@5 - Top-5 Precision, PR@AVG - Average Precision, MRR - Mean Reciprocal Rank, MicroF1 - Micro F1-score, MacroF1 - Macro F1-score, n.a - Not available)

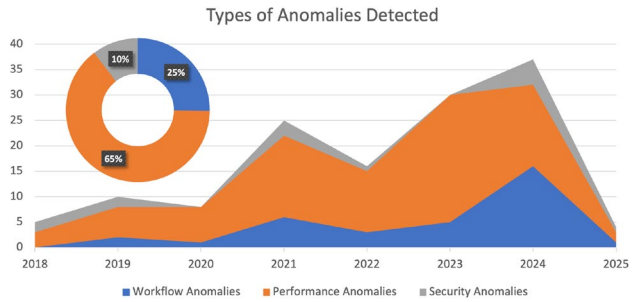

Fig. 5 Evolution of anomaly types detected

A novel approach was presented by Jacob *et al.* [88], who focused on anomaly detection for two types of attacks. The first type is a Password Guessing Attack, in which attackers repeatedly attempt to gain access to an application by trying every possible combination of usernames and passwords until one succeeds. The second type is SQL Injection, where malicious code is inserted into database instructions that rely on JavaScript instead of Structured Query Language (SQL) queries.

Baye *et al.* [95] centre their attention on the security of APIs because these components are easy targets for cyberattacks to access the system. Monitoring the traffic in the API layer can help identify several security menaces in the system.

Among the types of anomalies, performance anomalies stand out significantly from workflow and security anomalies (Figure 5). This is primarily because performance anomalies are a typology of anomalies directed at performance indicators and are thus apparently more easily identified in a system.

When comparing the categories of anomalies and the number of publications over time, an increase in workflow and security anomalies in the last years is apparent, thus showing a growing interest in this type of anomaly in general.

4.5 Root-cause identification

Detecting anomalies and diagnosing their origins in a microservices architecture approach is difficult and requires a lot of time [92]. In a microservices architecture, the distribution of services across multiple hosts makes their interconnections intricate and subject to constant change.

Table 7 Works on Root-cause Identification in Microservices

Works	Year	Root-Cause Identification Method [a]	Fault Type [b]	Performance (%) [c]
Wang *et al.* [189]	2018	G	n.d	n.a
Brandón *et al.* [1]	2020	ML/G	n.d	P: 83.9 - 70,2
Ji *et al.* [190]	2020	G	n.d	n.a
Qiu *et al.* [41]	2020	G	n.d	P: >80 R: >80
Wu *et al.* [188]	2020	G	n.d	n.a
Cai *et al.* [191]	2021	G	n.d	A: 93
Sun *et al.* [192]	2021	S	P	n.a
Xie *et al.* [193]	2023	G	P	A: 67.7 & 55.6
Yao *et al.* [194]	2023	G	n.d	MRR: 74.7
Zhang *et al.* [187]	2024	S	P	MRR: 70.6

[a]Root-Cause Identification Method (viz., ML - Machine Learning Methods, G - Graph Methods, S - Statistical Methods, n.d -Not described). [b]Type of Anomalies identified (viz., W - Workflow anomalies, P - Performance anomalies, S - Security anomalies). [c] P of the Algorithm (viz., PFR - False Positive Rate, A - Accuracy, P - Precision, R - Recall, F1 - F1-score, S - Specificity, MRR - Mean Reciprocal Rank, n.a - Not available)

Communication between services is possibly asynchronous (as in a message proxy) or synchronous (as in a direct request). System breakdown makes it difficult to determine the service correlation topology.

Additionally, microservices generate a significant amount of monitoring data. Analyzing and comprehending these indicators requires effort, and the results of simple procedures such as thresholding schemes are frequently incorrect. Not many algorithms are efficient in dealing with enormous amounts of monitoring data. The microservices architecture incorporates different cloud design patterns to improve stability and usability [6]. These patterns alter the way anomalies spread, making typical diagnostic methods ineffective.

As a result, anomaly detection algorithms will be required to spot and deal with these defence measures. How should one respond to a detected microservice anomaly? Because of the inherent and ever-changing nature of microservices, making an informed choice as to what the next steps in the recovery process should be is not easy. In addition, the study of performance anomaly detection and root-cause identification (RCI) often produces false positives.

An incorrect analysis leads to the wrong corrective action, decreases the chances of selecting the best recovery actions, and increases the risk of selecting the wrong steps. Furthermore, microservices are regularly updated, obliterating the historical data used to identify sources, reducing the accuracy of traditional data-driven methodologies, and increasing the complexity of the action selection.

Owing to the scalability of microservices, the number of monitoring metrics is quite large, resulting in substantial overhead and latency if all these data are utilized to select actions. Given the intricate nature of the infrastructure and microservices, the system operates with substantial uncertainty. Consequently, accurately forecasting the impact of recovery efforts is challenging [188].

The performance of upstream services directly influences the performance of downstream services. This interdependency implies that a problem arising in one upstream service can potentially cause multiple issues in downstream services. This means that RCI can have many potential origins [80].

Tables 6 and 7 compile information on the Root-Cause Identification approaches. Most of the proposed systems combine RCI with Anomaly Detection, although some studies only focus on identifying the causes of failures.

4.5.1 Machine learning techniques

Machine learning (ML) methods are widely applied to anomaly detection and root-cause identification in microservices, offering flexibility and the ability to learn from complex data. Classical models include decision trees, as used in DOCKERANALYZER [100], and one-class SVMs, applied to predict anomalous response time variations with high interpretability and generalization [7]. Other statistical–ML hybrids leverage RPCA, Isolation Forests, LOF, and OC-SVM for invocation chain and single-indicator anomaly detection [94]. These techniques remain attractive due to their simplicity, though scalability and interpretability challenges limit their adoption in production systems [1].

Graph-based and random walk methods represent another class of ML approaches. ServiceRank [92] infers service impacts without requiring predefined topologies, while MicroDiag [97] and AutoMAP [96] apply random walk models over service dependency graphs to replicate troubleshooting procedures and localize root causes. Bayesian reasoning has also been explored: GRALAF [126] employs a Causal Bayesian Network to capture relationships between fault states and service performance, identifying the most probable root cause. Mutual Information has been used to link anomaly scores with performance metrics [68], while DLA [106] emphasizes metric-level abnormality detection for root-cause inference.

More advanced ML techniques extend into predictive and deep learning. Zhou *et al.* [137] built anomaly prediction models using Random Forests, KNN, and MLP, with MLP achieving the best performance. Data augmentation strategies with oversampling and Gaussian noise have also been applied to address anomaly imbalance [68]. Deep learning frameworks incorporate graph structures and sequential data: DiagFusion [107] uses Graph Neural Networks to model fault propagation across microservices, while others exploit system logs and traces to train models capable of forecasting abnormal future behaviors.

Overall, machine learning approaches span classical models, graph-based algorithms, and deep neural networks. While they provide powerful tools for anomaly detection and root-cause analysis, they face challenges of explainability and data availability in fast-evolving microservice environments [1].

4.5.2 Graph-based techniques

Graph-based approaches are widely applied to anomaly detection and root-cause identification in microservices, as they capture the complex interdependencies among services and enable structured analysis of propagation patterns [1]. The dynamic topology of microservice systems often reveals structural shifts during anomalies, making graphs a natural choice for modeling causality and dependencies.

Early efforts constructed correlation and causal graphs. Lin *et al.* [130] applied the PC algorithm to generate correlation graphs, while Qiu *et al.* [41] proposed a knowledge-graph based causality discovery framework. TraceAnomaly [78] localized faults by comparing call paths within traces, and CloudRanger [189] modeled impact graphs inspired by operator behavior. These approaches showed how directed acyclic and impact graphs can formalize service correlations for root-cause reasoning.

Random walk techniques extend graph analysis by traversing service dependency graphs heuristically. Systems such as MS-Rank [56, 57], MicroDiag [97], and AutoMAP [96] use forward, backward, or self-transitions to replicate troubleshooting and prioritize highly correlated anomalies. MicroRCA [98] isolates anomalous subgraphs and applies personalized PageRank (PPR) [195] to locate faulty services, while MicroRank [80] integrates spectrum analysis with random walks to refine rankings.

Service dependency and propagation graphs provide richer context. MicroHECL [7] builds service call graphs with quality metrics to track propagation chains, and MFRL-CA [62] constructs Microservice Fault Propagation Graphs (MFPG) to perform iterative random walk scoring. Local dependency graphs have also been used for backward BFS traversal and anomaly chain reconstruction [65]. Cai *et al.* [191] combined service dependency and deployment graphs in ModelCoder to distinguish known from unknown faults, while AAMR [103] ranked anomalies in Service Dependency Graphs (SDGs) with personalized PageRank.

More recent frameworks apply advanced graph learning. Ji *et al.* [190] combined Granger causality tests with Attention LSTMs, Xu *et al.* [184] proposed Relational Graph Convolutional Networks over Heterogeneous Failure Dependency Graphs, and DiagFusion [107] employed Graph Neural Networks for propagation pattern learning. Knowledge graphs have also been adopted, as in MicroKGCL [194], which embeds multimodal feedback to match anomalies with historical cases, and ImpactTracer [193], which combines propagation modeling with backward tracing to score suspicious services.

While graph-based methods capture structural and causal relationships effectively, they may be overly complex for isolated failures (e.g., single node anomalies) where simpler techniques may be more efficient [1]. Nonetheless, their ability to model service dependencies and propagation makes them among the most powerful approaches for root-cause analysis in large-scale microservice environments.

4.5.3 Statistical techniques

Statistical methods are widely used for anomaly detection and root-cause identification, often by ranking services or operations according to suspiciousness scores derived from system metrics or traces. For example, Li *et al.* [55] prioritized service tags by frequency and indicator scores, while PageRank-style approaches such as MicroRank [80] and the PRV method [160] assigned weights to operations or service nodes to iteratively refine rankings. Similarly, Li *et al.* [74] calculated suspicious scores for sets of microservices, accounting for cases where only specific invocation paths exhibit failures.

Spectrum-based and sequential statistical methods extend these ideas. Ye *et al.* [81] applied Spectrum-Based Fault Localization (SBFL) to rank potential faulty elements, while TraceContrast [187] mined contrast sequential patterns, identifying event sequences frequent in anomalous but rare in normal traces. These techniques combine statistical scoring with structural features of traces to improve localization accuracy.

Reducing false alarms is also a focus of statistical approaches. WinG [158] filters positive detections by evaluating short anomaly intervals and adjusts ranking scores to suppress spurious results. Other models integrate additional data sources to refine results, such as Sun *et al.* [192], who constructed dependency trees from resource metrics and modeled propagation with Markov chains.

Finally, hybrid methods combine statistical ranking with graph-based reasoning. MicroNet [132] builds operation-centric dependency graphs and applies personalized PageRank to distinguish between inner abnormalities (execution-level) and outer abnormalities (interaction-level). These approaches highlight the versatility of statistical analysis, from simple ranking to hybrid graph-statistical models, providing interpretable and computationally efficient solutions for anomaly localization.

Table 8 Testbed Platforms Used on Publications Included in the Survey

Name	Source	Dataset Availability	Technologies	Works
ACME Air	[196]	Public	Java/Liberty	[99]
Alibaba EagleEye	-	Proprietary	n.a	[7]
BookInfo	[197]	Public	Python, Java, Ruby, Node.js	[82]
Ciena Co.	-	Proprietary	n.a	[139]
Cinema	-	Proprietary	n.a	[133]
Death Star (Social Network)	[198]	Public	Docker Over AWS	[14, 75, 84, 88, 171]
E-Shopper	[199]	Public	Java	[144]
Grid5000	[200]	Proprietary	n.a	[1]
Hipster-shop	[201]	Public	Java, Python, Node.js, Go, and C#	[80, 110, 122, 142, 193]
HUAWEI	-	Proprietary	n.a	[104]
IBM Bluemix	-	Proprietary	n.a	[56, 57, 65, 92, 96, 130, 134, 189]
Meituan	-	Proprietary	Raptor	[131]
Online Boutique	[202]	Public	Python, Go, C#, HTML, Javascript	[72, 103]
OpenStack	[203]	Public	OpenStack + Kubernetes	[85, 156]
Pymicro	[204]	Public	Phyton	[65, 92, 96, 134, 189]
SAVI	-	Proprietary	OpenStack + Kubernetes	[93]
Sock Shop	[205]	Public	Kubernetes + Prometheus	[41, 44, 62, 63, 89, 97, 98, 103, 137, 143, 159, 160, 163, 164, 168, 175, 179, 188, 192]
Stan's Robot Shop	[206]	Public	Javascript, PHP, Java, Python	[154]
Tencent Testbed	-	Proprietary	Java/PHP/Microservices	[179]
TPC-W	[207]	Public	Kubernetes	[14, 106]
Train Ticket	[208]	Public	Java, Node.js, Python, Mongo DB, MySQL DB	[11, 69, 74, 82, 83, 89, 108, 112, 114, 119, 121–125, 135, 137, 138, 141, 144, 184, 187, 190] [153, 175, 179, 185]
Virtual War Room	[118]	Public	Go, Python, HTML, Shell	[72]

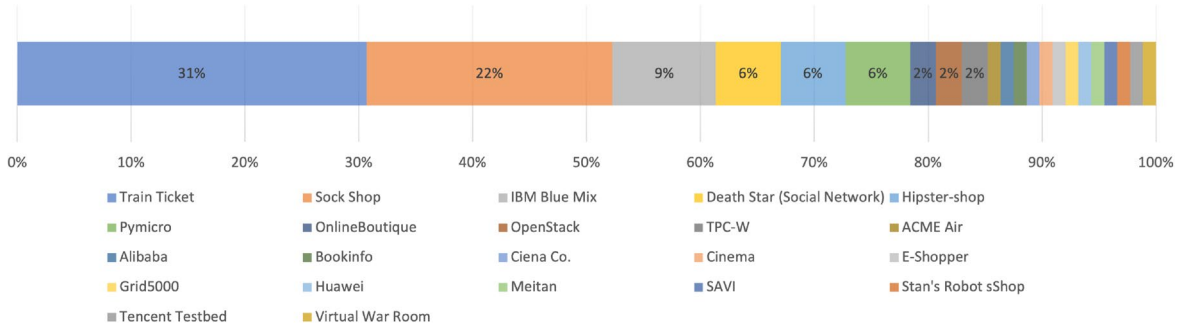

Fig. 6 Testbed platforms usage percentage

4.6 Testbeds and datasets used

The various publications included in this review demonstrated the validity of their findings through rigorous testing of both test and production microservice systems and platforms. These real-world evaluations provide a comprehensive assessment of the effectiveness and practicality of these methods in actual operational environments. The choice of microservice systems and platforms used for validation varies across studies, reflecting the diverse landscape of software development practices. Table 8 identifies these platforms, showing a mix of proprietary and open-source solutions.

The utilization of proprietary platforms in these studies speaks to the involvement of industry partners and organizations that might have access to their in-house systems,

Table 9 Datasets Used on Publications Included in the Survey

Name	Source	Dataset Availability	Year	Publications
AIOps 2018	-	Public	2018	[104]
AIOps Challenge 2020 and 2021	[209]	Public	2020	[55, 72, 79, 80, 94, 155, 158, 182, 191]
AIOps 2022	-	Proprietary	2022	[108, 109, 115, 132, 147, 151, 175]
AssureMOSS	[210]	Public	2022	[86]
AWS-1800/900	-	Proprietary	2020	[118]
BlueGene/L - BGL	[211]	Public	2020	[87, 116, 117, 143]
China Mobile	-	Proprietary	2021	[80, 81]
eBay Traces	-	Proprietary	2020	[73]
Ecom App	-	Proprietary	2024	[165]
GAIA - Generic AIOps Atlas	[212]	Proprietary	2023	[76, 107, 114, 174]
GMAT	-	Proprietary	2018	[77]
HDFS	[213]	Public	2022	[116, 117, 120]
IBM Cloud Dataset	-	Proprietary	2019	[20, 56, 57, 65]
KubAnomaly	[214]	Public	2019	[66]
LAMP-1800-10	[215]	Public	2020	[118, 128]
Library App	-	Proprietary	2024	[165]
M3 Testbed (Azure, AWS)	-	Proprietary	2024	[146]
MicroCU	[216]	Public	2023	[161]
MSDS	[217]	Public	2019	[109]
NETFLIX-1800-10	[218]	Public	2020	[118, 128]
OpenStack	[219]	Public	2017	[116]
Sock-Shop Dataset	[205]	Public	n.a	[182]
SMD	[220]	Public	2023	[163, 182]
Syslrn	[221]	Public	2022	[70]
TraceRCA	[222]	Public	2021	[11, 74]
Yahoo	[223]	Public	2022	[104]

ensuring a practical and industry-relevant validation process. On the other hand, the integration of open-source platforms emphasizes the accessibility and reproducibility of research results, allowing other researchers and developers to verify and build upon the findings.

Alibaba [7], Netflix [128], IBM BlueMix [130], and eBay [73] are the most relevant and well-known proprietary platforms where the proposed methods were used and validated.

As shown in Figure 6, the most commonly used open-source platform is Train Ticket, which consists of 41 microservices and utilizes technologies such as Java, Node.js, Python, MongoDB, and MySQL. Another notable platform is Sock Shop, developed by Google using Kubernetes and Prometheus. Despite its positive reception and widespread use within the scientific community, the Sock Shop project has recently been marked as deprecated by its authors. Another platform that incorporates multiple technologies is Hipster Shop, a fork of Google Sock Shop, developed in several programming languages, including Java, Python, Node.js, Go, and C#. Other platforms like PyMicro, Hipster Shop, and Death Star (Social Network) are also used in some publications.

In Table 9, we compile all the identified datasets, defining their sources and the publications that use them. Some datasets are not publicly available; therefore, we cannot determine the source in this case. In contrast, the other datasets are publicly available to the research community.

The datasets from the AIOps Challenge 2020 and 2021 [209] were the most cited, with nine publications referencing them. Public datasets such as HDFS [213], TraceRCA [222], and Yahoo [223] are also widely used. The most recent datasets, all published or created in 2023, include GAIA - Generic AIOps Atlas [212], MicroCU [216], and SMD [220]. The source codes for several algorithms are available for further use and investigation: TraceRCA [222], AutoLog [224], CIRCA [225], MicroCU [216], and CausalRCA [226]. Additionally, the IBM Cloud Dataset is utilized in various articles, and an important aspect for future research is the ability to compare methods, which makes the availability of algorithms crucial.

4.7 Methods comparison

To determine the type of method with better performance, we selected those with more than 80% values in the most frequent metrics (accuracy, F1-Score, precision, and recall). Based on the acquired values, the average and standard deviation of these values were calculated to obtain the mean performance.

By comparing the mean performance of the Data Collection Methods, we can observe that Log Based is the most accurate method for collecting data, with the highest accuracy, precision, and recall. Log/trace-based has the highest F1-Score, which means it is the most balanced method for collecting data, considering both precision and recall. Log/trace/monitoring has the lowest accuracy, precision, and recall and is the least accurate method for collecting data. Monitoring-based and Tracing-based methods have similar accuracy, precision, and recall and are both effective methods for collecting data, but are overcome by Log-based methods.

By analyzing the mean performance of different anomaly detection methods, Supervised Learning, Statistical, Trace Comparison, and Unsupervised Learning were evaluated based on several metrics: precision, Recall, F1-Score, and accuracy. Among these methods, statistical methods stand

out with an accuracy of 99.2%. On the other hand, Trace Comparison exhibited a recall of 99.0%, a Precision of 92,0%, and an F1-Score of 98.2%, the most significant values observed.

Supervised Learning methods show relatively high precision, F1-Score, and accuracy. However, the recall is slightly lower, indicating that this methods may have missed some anomalies. Statistical analysis methods demonstrated a high recall (93.0%), indicating their ability to detect most actual anomalies, and they also exhibited a high accuracy of 99.2%. Nonetheless, the precision and F1-Score showed more variability, with a lower precision score than the other methods. The absence of a standard deviation for recall and accuracy suggests consistent performance across evaluations. Trace Comparison methods achieved the highest recall (99.0%), indicating their ability to detect almost all actual anomalies. They also maintain high precision, F1-Score, and accuracy, reflecting good overall performance, although there is relatively higher variability in precision compared to the other methods. Unsupervised Learning methods, in contrast, have low accuracy and recall but the highest precision, meaning they are effective at minimizing false positives while not being as adept at identifying true positives.

We analysed the mean performance of four root-cause identification methods: graph-based, machine learning, machine learning/graph-based, and statistical methods. The Machine Learning approach is the most promising, with a precision of 94.9% and recall mean value of 98.0%, accurately identifying true RCs while minimizing false positives, supported by a high F1-Score (99.0%). However, its accuracy (94.3%) was slightly lower, suggesting potential misclassification. In addition, there is some variability in the performance, as indicated by the standard deviation values for precision and accuracy. The Graph-Based methods demonstrated strong performance, with high precision (92.7%) and reasonable recall (89.7%), striking a good balance in RCI. The combination of Machine Learning and Graph-Based methods only has partial data, providing a precision of 87.2%, but the absence of other metrics makes its overall effectiveness unclear. The Statistical methods showed acceptable performance, with a precision of 85.0%, recall of 88.0%, and an F1-Score of 85,8%. Its accuracy of 99,0%, the highest value, indicates that it is a suitable method for identifying false positives. This means that it may be useful for identifying potential RCs but should not be used as the sole method for root-cause identification.

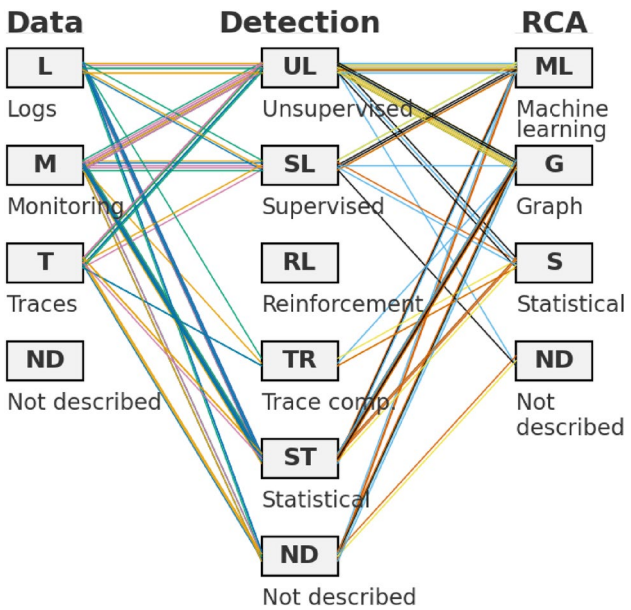

Fig. 7 Parallel-sets diagram mapping, for each work, the *Data Collection* method (L–Logs, M–Monitoring, T–Traces, ND–Not described) to the *Anomaly Detection* method (UL–Unsupervised, SL–Supervised, RL–Reinforcement, TR–Trace comparison, ST–Statistical, ND–Not described) and to the *Root-Cause Identification* method (ML–Machine Learning, G–Graph/causal, S–Statistical, ND–Not described). Each polyline represents a single paper flowing from Data → Detection → RCA

To guide the reader, Figure 7 provides a visual taxonomy that maps the main data sources (logs, traces, metrics) to the classes of anomaly detection techniques surveyed, and illustrates how these approaches integrate into root-cause analysis workflows. The diagram highlights three consistent patterns across the surveyed literature. On the *data* axis, traces (T) and monitoring (M) dominate over logs (L), reflecting the community's shift toward end-to-end causality and service interaction observability; nevertheless, several influential works still leverage logs, especially when system-level instrumentation is limited. On the *detection* axis, unsupervised learning (UL) forms the main corridor from T/M to RCA, with supervised methods (SL) appearing in scenarios with labeled faults and statistical or trace-comparison (ST/TR) persisting as lightweight baselines or when label scarcity precludes SL. On the *RCA* axis, graph approaches (G) attract most flows from both UL and SL, indicating that model-agnostic graph reasoning remains the prevailing strategy to rank suspects and propagate influence. ML-based RCA (ML) is also frequent, typically when the detection pipeline already extracts discriminative features, whereas purely statistical RCA (S) is less common and often paired with TR/ST detection. Two integration gaps also emerge: several works report strong detection performance but provide limited RCA specification (ND on the RCA axis), suggesting opportunities to couple anomaly scoring with explicit causal ranking and explanations; and a fraction of the corpus normalizes poorly to a strict detection taxonomy (e.g., graph-based "detection" labels), which we mapped conservatively for visualization. This underscores the field's methodological overlap between representation (graphs), inference (learning), and attribution (RCA).

Overall, the flows consolidate the current consensus: traces and monitoring feed predominantly unsupervised detectors, which then integrate with graph-centric RCA to produce ranked suspects and interpretable explanations.

5 Discussion

In this section, we present an overview of the most important aspects of this research and answer the RQs formulated at the beginning of the process.

5.1 RQ1: What are the specifics of anomaly detection in microservices architecture?

Distributed architectures, particularly microservices, introduce new challenges in the maintenance process owing to the various components of the system in a distributed platform and the constant changes in platform organization and active services. In this type of architecture, instances of a specific microservice can be replicated or terminated based on the immediate requirements of the system, requiring constant updates to the system status.

The health of a growing microservice system depends on a large part of its flawless functioning, which is crucial to user experience. Complexity increases over time, with numerous upgrades and features, and manual monitoring falls considerably short. The necessity of real-time information based on the instant state of the system is a reality in critical business applications; therefore, adopted methods or methodologies must be lightweight in terms of data processing and comprehensive to include all the relevant sources of information and data.

The results also need to be validated by DevOps operators; therefore, the method needs to be self-explainable. Decisions made using machine learning algorithms may be difficult, if not impossible, to explain at all. This is especially vital in RC analysis for mission-critical production systems because a single incorrect choice made by an automated system might go unreported and result in significant losses. Users must understand the causes of a model error to prevent and avoid repeating the same mistake in the future [1].

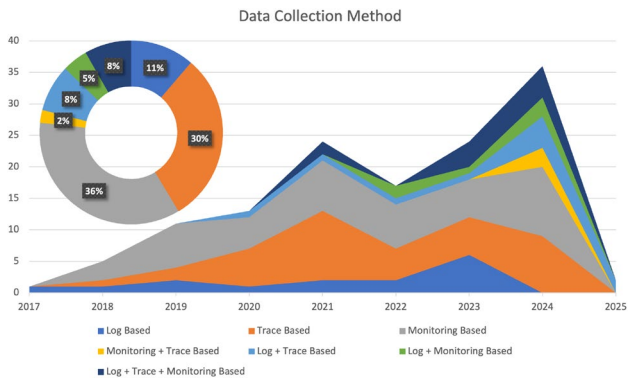

Fig. 8 Data collection methods used by algorithms

Microservices systems produce enormous amounts of monitoring and registry data; therefore, big data tools and platforms are needed to store the produced information. Machine learning algorithms need massive data to uncover the connections between metrics underlying a specific anomaly.

It is necessary to consider the time dimension so that the system development over time can be compared rather than comparing single snapshots. When working with microservices, containers are created and destroyed based on specified triggers and system instant demands, and the analysis of the system evolution is critical.

5.2 RQ2: Which approaches can be used for anomaly detection in microservices?

Several approaches have been applied to platform efficiency, performance, and resilience: the use of external systems to constantly test and probe services, the development and improvement of container orchestrators, and the ability to monitor the system to identify failures with the most precocity possible.

At the most basic level, anomaly detection is often accomplished by collecting historical data, establishing a baseline from the gathered data, and analyzing future data to discover any departures from the established baseline. An anomaly is defined as any divergence from the norm; when it comes to detecting abnormalities in microservices, this method is ineffective. Event information must also be considered in anomaly detection [227].

5.2.1 Collecting and gathering data

Monitoring-based methods take the lead in terms of data-collection methods. However, trace-based techniques have dominated in the last three years owing to the increased interest in these methods and their advantages in identifying complex anomalies (Figure 8). Additionally, the ability to make a more precise root-cause identification because of the more effective discovery of faulty microservices is crucial for the adoption of these methods.

Based on the data collection method, most algorithms gather information by monitoring several indicators from the system. This enables them to grab data in real-time and allows more rapid identification of an abnormal service or indicator.

In recent years, there has been a growing body of literature concentrating on traces. This interesting issue provides

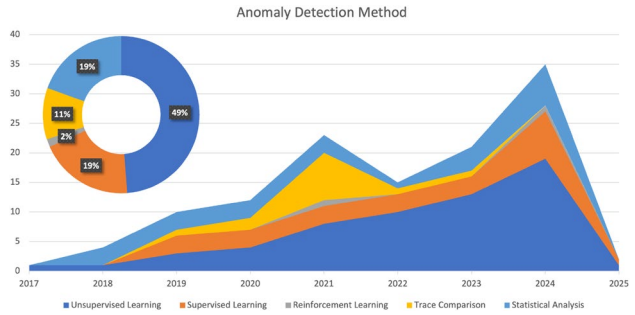

Fig. 9 Anomaly detection methods identified in this survey

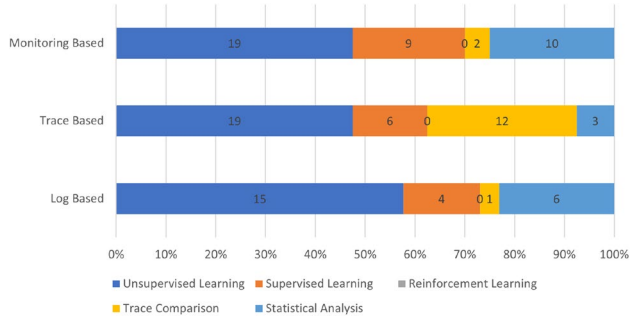

Fig. 10 Relationship between data collection method and anomaly detection technique

an opportunity to extend the knowledge and enhance some of the algorithms and methods used to deal with traces.

More recent works have introduced the combination of different data sources to feed the anomaly detection modules, namely, information from logs and traces, gaining some representation over single-source methods. Log analysis combined with monitoring performance indicators [160] or log and trace analysis [141] are recent trends in data acquisition algorithms. The approach proposed by Hou *et al.* [155], Bento *et al.* [154], Huang *et al.* [109], Ren *et al.* [108], and Gu *et al.* [131] combined these three data-collection methods to create a system with multiple data collections.

Observable system data describing the status of a distributed IT system is categorized into three categories based on their origin: performance metrics, logs, and distributed traces. Utilization and availability of data in the time series form include disk, CPU, network, memory, and service call response. Logs keep track of the activities performed by the application during execution. It is a kind of oriented metrics and log data source where data is specific to a particular service or resource, so it cannot track component interconnections of a distributed system.

In the end, the distributed traces form a graph-like representation of logs that illustrate the various services involved in responding to a specific user request. The traces comprise distinct occurrences or time periods. The spans store data on the execution process and performance of the microservices [85]. They maintain the information necessary for interservice communication and are more suited to activities such as anomaly detection. Implementing, collecting, and managing traceable data is substantially more complicated.

5.2.2 Detecting anomalies

The scientific community has used machine learning techniques in most publications when referring to anomaly detection in microservices, representing 70% of the methods used (Figure 9) with the expected growth in unsupervised methodologies in the previous year. Another predicted increase can be observed in trace comparison methods or trace-based sources.

Reinforcement learning is a recent subject in anomaly detection in microservices and has been presented in only one publication. On the other hand, unsupervised learning is the most used method. Several authors defend unsupervised learning because of its characteristics and ability to process information in systems with dynamic behaviour, such as a microservice infrastructure.

In addition, over the last two years, the use and relevance of analysing system traces for anomaly detection have increased in publications.

When analysing the relationship between data collection methods and anomaly detection methods (Figure 10), it is evident that machine learning techniques are typically used with monitoring or log-based data collection methods.

However, most trace-based methods are processed through trace comparisons, although machine learning and statistical analysis methods are gaining attention within the community.

There is no universally applicable outlier detection approach [43]. While novelty detection or novelty recognition appears to be a suitable method for identifying anomalies, the most appropriate method can only be determined through careful result analysis.

The failure identification process can be divided into three significant steps: Data Collection, Anomaly Detection, and root-cause Identification. Several published works include all three methods in their approach, whereas others focus their attention on some of them.

Data Collection involves identifying, capturing, and processing the system data. Three types of processes can perform this: log-based, where the data from log systems is collected; trace-based, where the traces of various requests are collected; and monitoring-based, where various performance indicators are collected in real time. All of these sources of information require specific treatment and have specific issues and technological approaches.

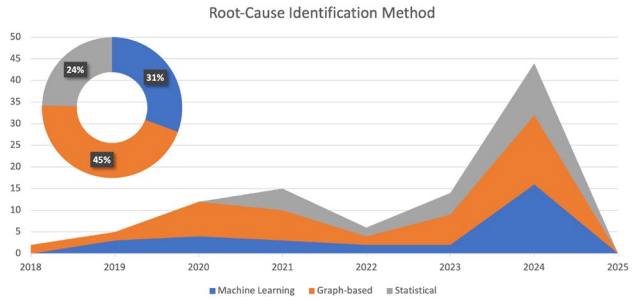

Fig. 11 Root-Cause identification methods

The worth of the collected information is a primary factor; therefore, combining sources can enhance data quality [131] by collecting performance indicators from logs or real-time monitoring, but also from traces, which is an important source of anomaly information in a microservices architecture.

Following Data Collection, the process of Anomaly Detection grabs the treated information from the previous cycle and detects abnormal behaviours with more significant accuracy, avoiding the risk of false-positive or false-negative values. Machine learning techniques, such as unsupervised, supervised, and reinforcement learning, dominate this step, mainly because of the large amount of data needed to process and the constant change in the system structural environment.

The comparison of distributed traces and the use of statistical methods are also techniques used to detect outliers or anomalies. The introduction of distributed trace analysis has greatly enhanced the identification of workflow failures specific to microservices. These techniques should be integrated with traditional monitoring methods to improve the performance and accuracy of anomaly detection in the system.

This study provides several data sources, such as logs, traces, and monitoring. Combining these three data sources will help develop a multi-origin data system with better and more accurate identification of RCs. Conversely, a system with more distinct sources increases the process complexity.

5.3 RQ3: How can the causes of these anomalies be identified?

The first root-cause and anomaly detection methods are based on collecting information from logs or analyzing performance by monitoring performance indicators. These methods were improved by introducing application-layer traces, which allowed the identification of specific anomalies encountered in a microservice system. They enhanced the global performance of the methods, enabling faster identification of anomalies and their primary cause of failure.

As shown in Figure 11, the RCI methods are dominated by graph-based methods because of the processing of impact graphs, service graphs, and traces generated by the anomaly detection methods. In recent years, statistical methods have emerged, representing 24% of all the methods used to identify RCs.

The techniques used for root-cause failure identification can be divided into four groups:

- **Rule-based techniques:** Knowledge from experts is converted into IF-THEN rules, which are matched to the system's behaviour. This process is easy to deploy but requires extensive knowledge of the system, making it a challenging and time-consuming task. In dynamic infrastructures like microservices, any system rearrangement will necessitate additional updates to the defined rules;
- **Learning-based techniques:** Rules are defined without human interaction, as the system is treated as an agnostic application or black box. A huge amount of data is required to train the model, posing an extra challenge due to the constant state changes and organizational shifts in microservices;
- **Case-based techniques:** Utilize earlier failures as cases and compare new faults with past instances. This approach effectively avoids repeating previous errors and adapts to system changes. However, due to the variety of technologies and frequent upgrades, the same issues in microservices often manifest differently, making case matching complicated and error-prone. Additionally, the large number of accessible metrics in microservices adds substantial complexity and latency to estimating their similarity;
- **Model-based methods:** Simulate various parts of the healing process, such as fault characteristics and properties of the actions, or use theoretical methodologies like Markov decision theory.

Central monitoring systems are complicated and costly to implement because of the need to collect data continuously; therefore, monitoring systems will have to collect data from various and diverse sources to improve system knowledge and information.

Knowledge of the system state is vital in an ever-changing system; however, it is a mutating reality. Microservices are systems that are evolving unceasingly in terms of structure, number of active services, and service programming versions. This evolution has led to the need to develop stateless systems that do not depend on preconceived system knowledge and strict rules. However, a prior understanding of the system, acquired by user experience or historical data, may be included in the process.

5.4 RQ4: What is the performance of the various identified methods?

Various approaches to anomaly detection can be observed, but one factor is common to all publications: the difficulty in identifying these anomalies in a microservice environment.

Tables 5, 6, and 7 compile all information on the performance of the various techniques. A relevant aspect stands out: most of the anomaly detection methods have a high performance in the presented studies, with more than 90% reaching 98% in various cases.

When evaluating the performance of various methods for data collection, anomaly detection, and root-cause identification, the following findings can be drawn:

Log-based is the most accurate data collection method, the combination of Log and Trace-Based methods is the most balanced, and combining all the methods types (Log/Trace/Monitoring-Based) is the least accurate.

For anomaly detection methods, Statistical is the top method with a high F1-Score and accuracy, while Trace Comparison excels in Recall and Precision. Supervised Learning has high accuracy but low recall, whereas Unsupervised Learning shows high precision but low accuracy and recall.

When analyzing the methods individually, beginning with Supervised Learning methods, Chen *et al.* [104] applied Adaptive Pattern Learning, but provided only precision (82.5%). Huang *et al.* [105] used a Variational Auto-Encoder, achieving a balanced performance with Precision (89%), Recall (85%), and F1-Score (87%). Yang *et al.* [122] combined LSTM and GRU, resulting in notable improvements with a Precision increase of +25.5% and Recall increase of +7.6%. Samir and Pahl [106] utilized Hierarchical Hidden Markov Models (HHMM) and achieved high accuracy (97%).

Moving on to statistical methods, Pan *et al.* [134] employed the SPOT algorithm and Extreme Value Theory (EVT), but no other metrics were provided. Cao *et al.* [86] achieved high F1-Score (98.2%) and Accuracy (99.2%). Ye *et al.* [81] used Statistical methods and attained a Precision of 82%, Recall of 93%, and F1-Score of 87%. Neither of these approaches revealed the algorithms that were used.

In Trace Comparison methods, Chen *et al.* [62] applied a Microservice Fault Propagation Graph (MFPG) and observed a precision increase of +12.75%. Meng *et al.* [14] utilized tree-matching and achieved high Precision (97%), Recall (99%), and F1-Score (98%). Cai *et al.* [79] used the SDG and VAE. Yu *et al.* [80] utilized a Trace Comparison method and saw a Recall increase of +22%.

For Unsupervised Learning methods, Xu *et al.* [63] applied K-means clustering and achieved a balanced Precision, Recall, and F1-Score (91%). Chen *et al.* [121] employed LSTM and an attention mechanism and achieved a Precision of 71.7%, a high Recall of 99.5%, and an F1-Score of 83.34%. Baye *et al.* [95] utilized an SVM and obtained an F1-Score of 96.4%. Ghorbani *et al.* [111] applied Autoencoders and achieved a Precision increase of +9% and a Recall increase of +14%. Wu *et al.* [97, 98] used distance-based online clustering BIRCH and attained high accuracy (97%) but did not provide other metrics. Zhang *et al.* [103] employed the same distance-based online clustering BIRCH method but provided only the Accuracy (94%). Zhang *et al.* [83] used the Graph Attention Network (GAT) and achieved balanced Precision (90.5%), Recall (98.7%), and F1-Score (94.4%). Chen *et al.* [11] utilized a VGAE and an LSTM-AE, achieving high Precision (97.6%) and Recall (93%), but slightly lower F1-Score (95%). Liu *et al.* [78] employed deep variational Bayesian networks, resulting in balanced Precision (98%), Recall (97%), and F1-Score (97%). Li *et al.* [74] used Trace analysis and observed a significant increase in Precision (+44.8%). Yu *et al.* [82] utilized K-means clustering and achieved balanced Precision (90%), Recall (86%), and F1-Score (88%).

Based on the analysis of the individual anomaly detection methods, we can determine the top five methods:

- Tree-matching [14] (Precision: 97%, Recall: 99%, F1-Score: 98%);
- Deep Variational Bayesian networks [78] (Precision: 98%, Recall: 97%, F1-Score: 97%);
- Support Vector Machine [95] (F1-Score: 96.4%);
- Variational graph auto-encoder and an LSTM autoencoder [11] (Precision: 97.6%, Recall: 93%, F1-Score: 95%);
- Graph Attention Network [83] (Precision: 90.5%, Recall: 98.7%, F1-Score: 94.4%).

In terms of root-cause identification, Machine Learning is observed to be the most promising root-cause identification method with excellent precision and recall, followed closely by graph-based methods. The combination of Machine Learning and Graph-Based methods requires further evaluation owing to limited data, and the statistical method performs reasonably well with high accuracy but lacks comprehensive assessment without F1-Score data.

Looking at these methods individually and starting with graph-based methods, Chen *et al.* [62] used a Microservice Fault Propagation Graph (MFPG) but did not provide Precision, Recall, or F1-Score. Wu *et al.* [97] employed a metric causality graph, achieving a high precision of 97%, but did not provide a Recall or F1-Score. Zhang *et al.* [103] used an SDG for performance-related faults but did not report the Precision, Recall, or F1-Score. Qiu *et al.* [41] utilized the PC algorithm for unspecified fault types and achieved a

Precision, Recall and F1-Score of 80%. Cai *et al.* [191] also employed the SDG with an Accuracy of 93%.

Regarding Machine Learning methods, Jin *et al.* [94] used robust principal component analysis (RPCA) for performance-related faults and achieved a precision of 90%, but the recall was not provided. Ma *et al.* [96] employed a Machine Learning method, achieving a 90% accuracy, mentioning the use of random walk. Zhou *et al.* [137] used a multilayer perceptron (MLP) for workload-related faults and achieved high precision (99.8%), recall (98%), and F1-Score (99%).

In statistical-based methods, Zhang *et al.* [160] utilized the PageRank Value (PRV) for workload-related faults and achieved a precision of 86% and recall of 83%. Ye *et al.* [81] used spectrum-based fault localization (SBFL) for unspecified fault types and achieved a precision of 82% and recall of 93%. Yu *et al.* [80] employed the PageRank Scorer for Performance-related faults and achieved a recall increase of +22% but did not provide Precision or F1-Score. Yang *et al.* [158] used a Ranking Score for Performance-related faults and achieved a Precision of 87%, but did not provide Recall or F1-Score. Based on the analysis of the individual root-cause identification methods, we can determine the top five methods:

- Multilayer Perceptron (MLP) [137] (Precision: 99.8%, Recall: 98%, F1-Score: 99%);
- Metric causality graph [97] (Precision: 97%);
- Robust Principal Component Analysis (RPCA) [94] (Precision: 90%);
- Spectrum-Based Fault Localization (SBFL) [81] (Precision: 82%, Recall: 93%, F1-Score: 87%);
- PageRank Value (PRV) [160] (Precision: 86%, Recall: 83%, F1-Score: 84,5%).

Overall, it isn't easy to compare the performances of various methods, mainly because different datasets and testbeds are used to detect anomalies. In addition, the absence of certain performance metrics in some studies complicates the comparison process between methods. When identifying anomalies, the data are very imbalanced because an anomaly is not a regular occurrence in the data; therefore, normal values outweigh anomalous values by an enormous margin. Therefore, different data sources can contribute to the better or worse performance of an algorithm, although most recent publications compare their method with others published earlier. Sharing the method source code and algorithm details enabled this comparison and allowed research groups to internally compare the efficiency of different methods with one another.

The aspect we identified with a higher chance of future development and a focus of our investigation is root-cause identification, where the performance indicators show lower performance. However, only a few publications exist on root-cause identification.

5.5 RQ5: Which testbeds or datasets can be used in future investigations?

In our study, we identified diverse microservice systems and platforms for validation, including a mix of proprietary and open-source solutions. Industry partners and organizations with access to proprietary platforms ensure practical and industry-relevant validation. Simultaneously, the integration of open-source platforms emphasizes the accessibility and reproducibility of the research results for other researchers and developers.

The most relevant and well-known proprietary platforms used for validation are Alibaba [7], Netflix [128], IBM BlueMix [130], and eBay [73]. The most commonly used open-source platform is Train Ticket [208], followed by Sock Shop [205], Hipster-shop [201], PyMicro [204], and Death Star (Social Network) [198], which have been used in various studies.

Table 9 lists the identified datasets, indicating the sources and publications that use them. Some datasets are not publicly available, limiting their accessibility, whereas others are available to the research community. Among the datasets, the AIOps Challenge 2020 and 2021 [209] published datasets are the most relevant and widely used, followed by the IBM Cloud Dataset in various articles. An important aspect for future investigation is the ability to compare methods, which makes algorithm availability crucial. The source codes of specific algorithms such as TraceRCA [222], AutoLog [224], CIRCA [225], MicroCU [216], and CausalRCA [226] are available for further utilization and investigation. This availability facilitates the assessment and advancement of anomaly detection techniques in the domain of microservices.

6 Challenges, open issues, and directions to future investigations

During the review process, various publications identified a number of challenges. In this section, we compile the most important of these challenges, highlight unanswered questions, and provide guidance for future investigations.

Logs introduce an extra challenge to developers, as the parser that grabs and processes data is configured with a determinate structure. System updates can change the message structure, causing failures in the entire anomaly detection system.

Various challenges arise with the usage of machine learning approaches in terms of anomaly detection and root-cause identification because microservice systems are constantly changing due to new versions of the system, such as rearrangement. Constant retraining of the model is necessary to incorporate these changes, causing massive processing demands to compute an updated model. However, when comparing the performance of a method that uses machine learning with those that do not, the latter has lower performance metrics, although they are faster, because there is no need to spend extra time training the model.

When applying machine learning methods to small- and mid-size microservice systems, training and retraining tasks can be performed more globally. In industrially distributed and different microservice systems, the behaviour of each virtual server differs from all the others; therefore, the training process is an extra task with significant implications for machine performance and optimization. Applying a global model to the whole infrastructure can be an exciting improvement in machine-layer maintenance, supervision, and optimization.

The use of reinforcement learning or processes that allow optimization of the monitoring system based on present behaviour and occurrences can be a challenge for future implementations because of the increasingly accurate knowledge of system behaviour. Microservices architecture may change over time due to alterations in the environment or user behaviour, so the data patterns acquired need to be updated. Another important approach is to embed adaptive pattern-learning algorithms to continuously learn and update their models as new data become available [104].

The various studies included in this survey make it possible to identify several unresolved issues and prospective research subjects that will be investigated further. Most articles refer to the will to extend the investigation to larger microservice systems to validate and confirm the results obtained in the testbed of the experiment [97]. For instance, ServiceRank [92] can also quickly analyse and diagnose anomalies in other involved systems, such as sensors and social and biological networks, opening the opportunity to apply these anomaly detection methodologies to different types of systems.

Another common approach to extending the investigation is validating the methodologies using diverse testbeds and datasets to ensure the accuracy of the collected data and enhance the method performance, precision, and adaptability to industrial applications.

6.1 Trusted execution environments for secure monitoring

Trusted Execution Environments (TEEs) [228], such as Intel SGX [229] or ARM TrustZone [230], are hardware-based solutions that provide isolated execution contexts for applications. In the context of microservices, TEEs can be used to ensure that monitoring agents, anomaly detection models, and root-cause analysis components execute in a secure and tamper-proof manner [231, 232, 232]. This guarantees trust in the monitoring pipeline, even in hostile or multi-tenant cloud settings [233]. Emerging use cases include secure log collection, trace verification, and confidential sharing of anomaly features across services, which can strengthen the reliability of detection frameworks while preserving privacy and integrity [234].

6.2 Trusted distributed AI for embedded microservices

Trusted Distributed AI extends the principles of federated and collaborative learning with trust guarantees, ensuring that distributed models can be trained and deployed without exposing sensitive data or compromising model integrity [235, 236]. For anomaly detection in microservices, TDAI is particularly relevant in edge and IoT scenarios, where embedded AI models are deployed close to the data source [237, 238]. By combining local inference with trusted aggregation, TDAI can enhance both privacy and robustness of anomaly detection pipelines [239]. Emerging applications include IoT-enabled monitoring platforms, distributed industrial control systems, and privacy-preserving RCA in smart manufacturing and healthcare microservices [240].

While most studies focus primarily on detection accuracy and algorithmic performance, trustworthiness is increasingly recognized as a fundamental requirement for dependable AI systems [53, 241, 242]. In distributed microservice environments, trust must be understood as a methodological evaluation property rather than a purely technological feature.

In this survey, trustworthiness is characterized across four complementary dimensions:

1. **Reliability** – stability of detection and root-cause results under varying workloads and noise, consistent with evaluation and production-readiness principles in distributed systems [50, 243];
2. **Explainability** – ability to justify diagnostic decisions through interpretable causal or statistical evidence, aligned with established XAI and causal reasoning frameworks [52, 242, 244–246];
3. **Consistency** – reproducibility of root-cause ranking across executions and environments, reflecting reproducibility principles in production fault diagnosis systems [50, 51];
4. **Robustness** – resistance to incomplete, noisy, or heterogeneous telemetry data, as emphasized in robustness benchmarking and AI safety research [53, 247, 248].

Table 10 Trust Dimensions and Operational Indicators in Distributed Microservice Diagnosis

Trust Dimension	Operational Indicator	Analytical Interpretation	Representative References
Reliability	Detection Stability Index (DSI)	Stability of anomaly detection and localization performance across workload variations and datasets	[22, 50]
Consistency	Root-Cause Ranking Variance (RCRV)	Reproducibility of root-cause ranking under repeated executions or environmental changes	[51, 80]
Explainability	Causal Interpretability Score (CIS)	Availability of interpretable causal or structural justifications supporting diagnostic decisions	[242, 244, 245]
Robustness	Noise Degradation Ratio (NDR)	Sensitivity of detection and localization performance to noisy, incomplete, or perturbed telemetry data	[53, 247]
Causal Validity	Fault Alignment Ratio (FAR)	Degree of alignment between predicted root cause and validated ground-truth or injected faults	[52, 246]
Scalability	Performance Scalability Index (PSI)	Preservation of diagnostic performance under service growth and large-scale distributed workloads	[78, 243]
Telemetry Integrity	Data Coverage Ratio (DCR)	Completeness and structural coherence of telemetry used in distributed diagnosis pipelines	[248, 249]

These dimensions provide a structured methodological perspective for evaluating trustworthy distributed diagnostic systems beyond purely technological comparisons.

Traditional Trusted AI focuses on the reliability, transparency, fairness, and robustness of models operating in centralized environments [53, 241, 242]. However, microservice-based systems are inherently distributed, dynamic, and partially observable, where diagnostic outcomes emerge from the interaction of multiple services, heterogeneous telemetry sources, and continuously evolving system topologies [243, 248]. This perspective aligns with emerging discussions on Trusted Distributed AI, which extend traditional Trusted AI principles to decentralized environments characterized by cross-service dependencies, partial observability, and dynamic topology evolution.

In microservices, anomaly detection and root-cause identification cannot rely solely on model-level trust [241, 242]. Trust must be established across the entire distributed diagnostic pipeline, including data collection, causal reasoning, and fault propagation analysis [52, 246, 248]. TDAI is therefore essential to ensure reliable and explainable diagnosis in large-scale cloud-native systems [53, 243].

Recent advances in anomaly detection and root-cause identification increasingly highlight the importance of trustworthiness in distributed diagnostic systems [230]. In microservice-based architectures, diagnostic outcomes depend on heterogeneous telemetry sources, dynamic service interactions, and evolving execution paths. Consequently, trust cannot be reduced to detection accuracy alone, but must incorporate reliability, consistency, interpretability, robustness, and causal soundness.

Prior research in anomaly detection [22], interpretable machine learning [242, 244, 245], robustness evaluation [53], and causal inference [52, 246] provides conceptual foundations for operationalizing trust in intelligent systems. Building upon these principles, Table 10 synthesizes measurable trust dimensions and relates them to diagnostic systems in microservice environments. From an operational perspective, several of these trust dimensions can be quantified using the performance metrics previously discussed in Section 3.8. For instance, reliability can be assessed through stability of Precision, Recall, and F1-score across datasets and workload conditions, while robustness can be evaluated through variations in False Positive Rate (FPR) and False Negative Rate (FNR) under noisy or incomplete telemetry. Consistency can be approximated through ranking stability metrics such as Mean Reciprocal Rank (MRR), and explainability through the presence of causal or structural justification mechanisms reported in the evaluated studies.

The operational indicators summarized in Table 10 do not introduce new evaluation metrics per se, but rather reorganize existing evaluation principles into a structured trust-oriented perspective. This perspective enables comparative analysis of distributed anomaly detection and root-cause identification systems beyond accuracy-centric evaluation, aligning with emerging discussions on trustworthy and dependable AI systems in distributed environments.

Formally, trust in distributed diagnostic systems can be viewed as a multidimensional construct T which is a function of reliability (R), robustness (Ro), consistency (C), and explainability (E), such that $T = f(R, R_o, C, E)$, where each component can be operationalized using measurable

performance indicators derived from classification and ranking metrics [241].

6.3 Directions for future research

Based on the analysis of the different publications on your research process, we can point out some directions for future works that intend to explore anomaly detection on the microservices theme, namely:

- Using different and diverse sources of information in the methods, including traces, KPI monitoring, logs, and combinations of these three data collection methods [85];
- Improving the efficiency of methods based on traces, logs, and metrics [7]. Some improvements can be made in method optimization, such as using feature selection to minimize metric dimensionality [97];
- Optimizations can also be achieved using less time-consuming methods, especially graph computation. Transforming graphs into vectors could also be a possibility, and in theory, vectors are a lighter data structure in terms of computing requirements [1];
- Extending tracing to additional parts of the system state and metadata such as logging and monitoring [71]. Linking more information to traces by adding metadata pieces of information to spans such as user identifiers or cookies [154];
- Although this can be problematic when changes in the microservice structure occur, the usage of historical data can also be an opportunity to improve the system knowledge of its behaviour. The development of a hybrid solution with a learning mechanism using past failure data [188] is a possible solution with positive results in anomaly detection, particularly in the process of RCI. The introduction of historical data is essential because the similarity between values of a measure must be computed differently based on how historical data is distributed [1]; thus, the past values must also be considered in the data models;
- Systems may evolve into online and immediate anomaly detection systems [94] because of their greater dependency on distributed systems in the future. The ability to process online data is an advantage of software platforms used by many users, thus reducing the time to identify faults or anomalies;
- Massive data production by these systems, it is essential to select and acquire only the necessary amount of data to decrease the system's needed resources and create a more straightforward approach in a manner that reduces the search space and hence the system's response time [1];
- Security issues are a vast area where these methods can be tested and validated, not just in terms of system performance (DDoS attacks), but also in identifying anomalous behaviours and workflows of users via trace analysis [88];
- The analysis of different microservice systems running on the same infrastructure can also be an interesting point for future study, mainly because of its advantages for mid-size platforms that do not have a dedicated infrastructure. One possible path is aggregating information based on (micro)service categories to improve performance on shared platforms [55]. This approach can also be used in large software architectures by grouping cluster containers running the same microservice;
- Apply different methods for identifying the causes of failures by comparing methods used in other fields of application rather than microservices;
- Creating intelligent and user-less systems will also be an interesting point of view regarding automatic configuration and reconfiguration, owing to some changes in the microservices structure, eliminating the need for user participation and setup [61];
- Regenerative AI (Artificial Intelligence) methods are used to perform a text-based analysis of data from log files, determine the typical values, and identify anomalies without prior knowledge of the system architecture. Training a generative model such as a VAE or a GAN;
- Creating a method that automatically selects the best indicators and data sources based on the actual data grabbed and the historical behaviour of these indicators will be a significant development, allowing the existence of self-monitoring systems. This approach is vital in serverless systems where containers are produced and destroyed based on events and the instant needs of the system [1].

6.4 Limitations of this survey

While this survey aimed to be as comprehensive as possible, several limitations must be acknowledged. First, the scope of the review was limited to studies published in English between 2012 and May 2025, which may have excluded works outside this timeframe or in other languages. Second, although seven major digital libraries were searched using systematic PRISMA guidelines, the possibility of selection bias cannot be entirely eliminated. Third, due to the heterogeneity of datasets, experimental setups, and evaluation metrics across the reviewed studies, direct performance comparisons should be interpreted with caution. Fourth, the field of anomaly detection in microservices is rapidly evolving, and newly published methods beyond our cut-off date are not captured here. These limitations suggest that, while

this article provides a representative and timely overview, it should be complemented with future updates as the field matures.

7 Conclusion

The microservice environment and the performance of microservice platforms continue to represent an area of rapid growth, driven by the need to ensure reliability and to protect both critical information and infrastructure against malicious activities and anomalies in service operations

Microservices also facilitate business management by aligning technological infrastructures with business processes. This approach allows organizations to form teams with strong knowledge of their business processes, covering development, deployment, and maintenance tasks more effectively.

At the same time, microservice systems have become increasingly vast and complex, making manual supervision insufficient. Production systems increasingly require AI-enabled anomaly detection combined with automated or semi-automated corrective actions. Current research shows promising advances in this direction, yet no single approach fully addresses the scalability, explainability, and real-time challenges of modern deployments.

Looking ahead, AI-enabled supervision appears to be a promising path, particularly when integrated with real-time prevention and remediation mechanisms. However, the practical deployment of such solutions must carefully account for limitations in dataset availability, heterogeneous evaluation setups, and the evolving landscape of trusted computing (e.g., TEEs and TDAI). Future research should move beyond purely performance-driven approaches and focus on trustworthy distributed diagnosis, where reliability, explainability, and robustness are explicitly quantified. The evolution from Trusted AI toward Trusted Distributed AI represents a key direction for enabling dependable anomaly detection and root-cause identification in large-scale microservice ecosystems. Organizations aiming to maximize the benefits of microservices and cloud-native systems can benefit from these techniques, but their effectiveness will depend on rigorous benchmarking, continued research, and careful integration into operational workflows.

Author Contributions All authors contributed to the conception and design of thestudy. Luís Barata collected and analysed the data, conducted the initial research, performed the investigation and methodology design, and wrote the main manuscript text, including the preparation of the first draft. Sérgio Sequeira reviewed the manuscript and provided grammatical improvements. Pedro Inácio, Eurico Lopes and Mário Freire supported the investigation process, guided the development of the research methodology, and contributed to the overall manuscript structure. Mário Freire and Pedro Inácio were responsible for funding acquisition, project administration, and assuring completeness, correctness, and neutrality. Mário Freire and Eurico Lopes were also responsible for supervising the research program. All authors reviewed, edited, and approved the final version of the manuscript.

Funding Open access funding provided by FCT|FCCN (b-on). This work is funded by national funds through FCT – Fundação para a Ciência e a Tecnologia, I.P., and, when eligible, co-funded by EU funds under project/support UID/50008/2025 – Instituto de Telecomunicações, with DOI identifier https://doi.org/10.54499/UID/50008/2025; in part by the project "T3SH City – CENTRO2030-FEDER-02586700 – Towards Truly Smart, Secure, Sustainable, and Healthy Cities", funded by the Centro 2030 Regional Programme; in part by the WATERMARK project (Watermark-Based Algorithms for Trustworthy Media Authentication and Robust Certification in Public Administration), Project No. 2024.07356.IACDC, supported by "RE-C05-i08.M04 – Support the launch of a program of R&D projects aimed at the development and implementation of advanced systems in cybersecurity, artificial intelligence, and data science in public administration, as well as a scientific training program," under the Recovery and Resilience Plan (PRR), as part of the funding agreement signed between the Recovery Portugal Task Force (EMRP) and the Fundação para a Ciência e a Tecnologia (FCT); and also in part supported by UID/04516/NOVA Laboratory for Computer Science and Informatics (NOVA LINCS) with the financial support of FCT.IP.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors have no relevant financial or non-financial interests to disclose.

References

1. Brandón, A., Solé, M., Huélamo, A., Solans, D., Pérez, M.S., Muntés-Mulero, V.: Graph-based root cause analysis for service-oriented and microservice architectures. Journal of Systems and Software **159**, 110432 (2020). https://doi.org/10.1016/j.jss.2019.110432
2. Flexera: Flexera 2024 State of the Cloud | Report (2024). https://info.flexera.com/CM-REPORT-State-of-the-Cloud Accessed 2023-12-05
3. Cai, L., Qi, Y., Wei, W., Li, J.: Improving resource usages of containers through auto-tuning container resource parameters. IEEE Access **7**, 108530–108541 (2019). https://doi.org/10.1109/ACCESS.2019.2927279

4. IBM: What is SOA (Service-Oriented Architecture) (2021). https://www.ibm.com/cloud/learn/soa Accessed 2021-05-24
5. Fetzer, C.: Building critical applications using microservices. IEEE Security & Privacy **14**(6), 86–89 (2016). https://doi.org/10.1109/MSP.2016.129
6. Newman, S.: Building microservices: designing fine-grained systems. O'Reilly Media. https://books.google.pt/books?id=jjl4BgAAQBAJ
7. Liu, D., He, C., Peng, X., Lin, F., Zhang, C., Gong, S., Li, Z., Ou, J., Wu, Z.: MicroHECL: High-efficient root cause localization in large-scale microservice systems. In: Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 338–347 (2021). https://doi.org/10.1109/ICSE-SEIP52600.2021.00043
8. Wang, R., Ying, S., Li, M., Jia, S.: HSACMA: a hierarchical scalable adaptive cloud monitoring architecture. Software Qual. J. **28**(3), 1379–1410 (2020). https://doi.org/10.1007/s11219-020-09524-z
9. Blog, N.T.: The Netflix Simian Army (2018). https://netflixtechblog.com/the-netflix-simian-army-16e57fbab116 Accessed 2022-01-05
10. Inc., D.: Docker: Accelerated, Containerized Application Development. https://www.docker.com/ Accessed 2022-05-28
11. Chen, J., Liu, F., Jiang, J., Zhong, G., Xu, D., Tan, Z., Shi, S.: TraceGra: A trace-based anomaly detection for microservice using graph deep learning. Computer Communications **204**, 109–117 (2023). https://doi.org/10.1016/j.comcom.2023.03.028
12. M.A, F.Y.E.: XLI. On discordant observations. The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science (2009). https://doi.org/10.1080/14786448708628471
13. Pang, G., Shen, C., Cao, L., Hengel, A.V.D.: Deep learning for anomaly detection: A review. ACM Comput. Surv. **54**(2), 38 (2021). https://doi.org/10.1145/3439950
14. Meng, L., Ji, F., Sun, Y., Wang, T.: Detecting anomalies in microservices with execution trace comparison. Futur. Gener. Comput. Syst. **116**, 291–301 (2021). https://doi.org/10.1016/j.future.2020.10.040. Accessed 2021-12-20
15. Rosà, A., Chen, L.Y., Binder, W.: An endpoint communication profiling tool for distributed computing frameworks. In: Proceedings of the 2016 IEEE 36th International Conference on Distributed Computing Systems (ICDCS), pp. 765–766 (2016). https://doi.org/10.1109/ICDCS.2016.67. ISSN: 1063-6927
16. Zhou, J., Chen, Z., Mi, H., Wang, J.: MTracer: A trace-oriented monitoring framework for medium-scale distributed systems. In: Proceedings of the 2014 IEEE 8th International Symposium on Service Oriented System Engineering, pp. 266–271 (2014). https://doi.org/10.1109/SOSE.2014.37
17. Wang, T., Zhang, W., Ye, C., Wei, J., Zhong, H., Huang, T.: FD4C: Automatic fault diagnosis framework for web applications in cloud computing. IEEE Transactions on Systems, Man, and Cybernetics: Systems **46**(1), 61–75 (2016). https://doi.org/10.1109/TSMC.2015.2430834
18. Wolfe, S.: Amazon's one hour of downtime on Prime Day may have cost it up to $100 million in lost sales. https://www.businessinsider.com/amazon-prime-day-website-issues-cost-it-millions-in-lost-sales-2018-7 Accessed 2021-11-24
19. Infopulse: The Importance of Microservices Architecture for Modern Applications. https://www.infopulse.com/blog/the-importance-of-microservices-architecture-for-modern-applications/ Accessed 2021-12-09
20. Islam, M.S., Pourmajidi, W., Zhang, L., Steinbacher, J., Erwin, T., Miranskyy, A.: Anomaly detection in a large-scale cloud platform. In: Proceedings of the 43rd International Conference on Software Engineering: Software Engineering in Practice. ICSE-SEIP '21, pp. 150–159. IEEE Press, Virtual Event, Spain (2021). https://doi.org/10.1109/ICSE-SEIP52600.2021.00024
21. Steinder, M.I., Sethi, A.S.: A survey of fault localization techniques in computer networks. Science of Computer Programming **53**(2), 165–194 (2004). https://doi.org/10.1016/j.scico.2004.01.010
22. Chandola, V., Banerjee, A., Kumar, V.: Anomaly detection: A survey. ACM Comput. Surv. **41**(3), 15 (2009). https://doi.org/10.1145/1541880.1541882
23. Akoglu, L., Tong, H., Koutra, D.: Graph based anomaly detection and description: a survey. Data Min. Knowl. Disc. **29**(3), 626–688 (2015). https://doi.org/10.1007/s10618-014-0365-y
24. Wong, W.E., Gao, R., Li, Y., Abreu, R., Wotawa, F.: A survey on software fault localization. IEEE Trans. Software Eng. **42**(8), 707–740 (2016). https://doi.org/10.1109/TSE.2016.2521368
25. Solé, M., Muntés-Mulero, V., Rana, A.I., Estrada, G.: Survey on models and techniques for root-cause analysis (2017). https://arxiv.org/abs/1701.08546
26. Soldani, J., Brogi, A.: Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey. ACM Comput. Surv. **55**(3) (2022) https://doi.org/10.1145/3501297
27. Zhang, S., Xia, S., Fan, W., Shi, B., Xiong, X., Zhong, Z., Ma, M., Sun, Y., Pei, D.: Failure diagnosis in microservice systems: A comprehensive survey and analysis. ACM Trans. Softw. Eng. Methodol. (2025). https://doi.org/10.1145/3715005
28. Wei, W., Liu, L.: Trustworthy distributed ai systems: Robustness, privacy, and governance. ACM Comput. Surv. (2025). https://doi.org/10.1145/3645102
29. A?Ca, M.A., Faye, S., Khadraoui, D.: Trusted distributed artificial intelligence (tdai). IEEE Access **11**, 113307–113323 (2023). https://doi.org/10.1109/ACCESS.2023.3322568
30. Aca, M.A., Faye, S., Khadraoui, D.: A survey on trusted distributed artificial intelligence. IEEE Access **10**, 55308–55337 (2022). https://doi.org/10.1109/ACCESS.2022.3176385
31. McIlroy, M.D., Pinson, E.N., Tague, B.A.: UNIX time-sharing system: Foreword. Bell System Technical Journal **57**(6), 1899–1904 (1978). https://doi.org/10.1002/j.1538-7305.1978.tb02135.x
32. Fowler, M.: Microservices Definition. https://martinfowler.com/articles/microservices.html Accessed 2021-05-29
33. Divante: 10 companies that implemented the microservice architecture and paved the way for others (2019). https://divante.com/blog/10-companies-that-implemented-the-microservice-architecture-and-paved-the-way-for-others/ Accessed 2021-05-30
34. Alshuqayran, N., Ali, N., Evans, R.: A systematic mapping study in microservice architecture. In: Proceedings of the 2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA), pp. 44–51 (2016). https://doi.org/10.1109/SOCA.2016.15
35. Villamizar, M., Garcés, O., Castro, H., Verano, M., Salamanca, L., Casallas, R., Gil, S.: Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud. In: Proceedings of the 2015 10th Computing Colombian Conference (10CCC), pp. 583–590 (2015). https://doi.org/10.1109/ColumbianCC.2015.7333476
36. Lewis, J., Fowler, M.: Microservices (2017). https://martinfowler.com/articles/microservices.html Accessed 2021-05-23
37. Richter, D., Konrad, M., Utecht, K., Polze, A.: Highly-Available Applications on Unreliable Infrastructure: Microservice Architectures in Practice. In: Proceedings of the 2017 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C), pp. 130–137 (2017). https://doi.org/10.1109/QRS-C.2017.28
38. Landim, L., Barata, L., Lopes, E.: A simple approach to detect anomalies in microservices-based systems using PyOD. In:

Proceedings of the 22. Conferência da Associação Portuguesa de Sistemas de Informação (CAPSI'2022), pp. 177–186 (2022)
39. Deploy, O.: Monoliths versus microservices. https://octopus.com/blog/monoliths-vs-microservices Accessed 2023-06-07
40. Bruce, M., Pereira, P.A.: Microservices in Action. Manning. https://books.google.pt/books?id=Gts-swEACAAJ
41. Qiu, J., Du, Q., Zhang, S.-L., Qian, C.: A causality mining and knowledge graph based method of root cause diagnosis for performance anomaly in cloud applications. Applied Sciences (2020). https://doi.org/10.3390/app10062166
42. Song, X., Wu, M., Jermaine, C., Ranka, S.: Conditional anomaly detection. IEEE Trans. Knowl. Data Eng. **19**(5), 631–645 (2007). https://doi.org/10.1109/TKDE.2007.1009
43. Hodge, V.J., Austin, J.: A survey of outlier detection methodologies. Artif. Intell. Rev. **22**(2), 85–126 (2004). https://doi.org/10.1007/s10462-004-4304-y
44. Cao, W., Cao, Z., Zhang, X.: Research on microservice anomaly detection technology based on conditional random field. J. Phys: Conf. Ser. **1213**(4), 042016 (2019). https://doi.org/10.1088/1742-6596/1213/4/042016
45. Karkouch, A., Mousannif, H., Al Moatassime, H., Noel, T.: Data quality in internet of things: A state-of-the-art survey. Journal of Network and Computer Applications **73**, 57–81 (2016). https://doi.org/10.1016/j.jnca.2016.08.002
46. Zhou, X., Peng, X., Xie, T., Sun, J., Ji, C., Li, W., Ding, D.: Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study. IEEE Transactions on Software Engineering, 1–18 (2018) https://doi.org/10.1109/TSE.2018.2887384
47. Developers, G.: Classification: Precision and Recall | Machine Learning Crash Course | Google Developers. https://developers.google.com/machine-learning/crash-course/classification/precision-and-recall Accessed 2022-03-20
48. Antunes, N., Vieira, M.: On the metrics for benchmarking vulnerability detection tools. In: Proceedings of the 2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, pp. 505–516 (2015). https://doi.org/10.1109/DSN.2015.30. ISSN: 2158-3927
49. Cunha, V.C., Zavala, A.Z., Magoni, D., Inácio, P.R.M., Freire, M.M.: A complete review on the application of statistical methods for evaluating internet traffic usage. IEEE Access **10**, 128433–128455 (2022). https://doi.org/10.1109/ACCESS.2022.3227073
50. Breck, E., Cai, S., Nielsen, E., Salib, M., Sculley, D.: The ml test score: A rubric for ml production readiness and technical debt reduction. In: 2017 IEEE International Conference on Big Data (Big Data), pp. 1123–1132 (2017). https://doi.org/10.1109/BigData.2017.8258038
51. Yuan, D., Park, S., Zhou, Y., Savage, S.: Sherlock: Performance bug diagnosis in production systems. In: Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2012), pp. 1–14. USENIX Association, Berkeley, CA, USA (2012)
52. Pearl, J.: Causality: Models. Reasoning and Inference. Cambridge University Press, Cambridge, UK (2009)
53. Hendrycks, D., Dietterich, T.: Benchmarking neural network robustness to common corruptions and perturbations. In: International Conference on Learning Representations (ICLR) (2019)
54. Barata, L.M., Sequeira, S., Lopes, E., Inácio, P., Freire, M.: Anomaly Detection in Microservices Survey GitHub repository - Luís Barata. Accessed: 2025-05-14 (2025). https://github.com/luisbarata-ubi/anomally-detection-survey
55. Li, M., Tang, D., Wen, Z., Cheng, Y.: Microservice anomaly detection based on tracing data using semi-supervised learning. In: Proceedings of the 2021 4th International Conference on Artificial Intelligence and Big Data (ICAIBD) (2021). https://doi.org/10.1109/ICAIBD51990.2021.9459100
56. Ma, M., Lin, W., Pan, D., Wang, P.: MS-rank: Multi-metric and self-adaptive root cause diagnosis for microservice applications. In: Proceedings of the 2019 IEEE International Conference on Web Services (ICWS), pp. 60–67 (2019). https://doi.org/10.1109/ICWS.2019.00022
57. Ma, M., Lin, W., Pan, D., Wang, P.: Self-adaptive root cause diagnosis for large-scale microservice architecture. IEEE Transactions on Services Computing, 1–1 (2020) https://doi.org/10.1109/TSC.2020.2993251
58. Bobel, M., Gerostathopoulos, I., Bures, T.: A toolbox for realtime timeseries anomaly detection. In: Proceedings of the 2020 IEEE International Conference on Software Architecture Companion (ICSA-C), pp. 278–281 (2020). https://doi.org/10.1109/ICSA-C50368.2020.00053
59. He, S., He, P., Chen, Z., Yang, T., Su, Y., Lyu, M.R.: A survey on automated log analysis for reliability engineering. ACM Comput. Surv. **54**(6), 130–113037 (2021). https://doi.org/10.1145/3460345
60. Catillo, M., Pecchia, A., Villano, U.: AutoLog: Anomaly detection by deep autoencoding of system logs. Expert Systems with Applications **191**, 116263 (2022). https://doi.org/10.1016/j.eswa.2021.116263
61. Dragan, I., Iuhasz, G., Petcu, D.: A scalable platform for monitoring data intensive applications. Journal of Grid Computing **17**(3), 503–528 (2019). https://doi.org/10.1007/s10723-019-09483-1
62. Chen, Y., Chen, N., Xu, W., Lian, L., Tu, H.: MFRL-CA: Microservice fault root cause location based on correlation analysis. In: Proceedings of the 2021 8th International Conference on Dependable Systems and Their Applications (DSA), pp. 90–101 (2021). https://doi.org/10.1109/DSA52907.2021.00018. ISSN: 2767-6684
63. Xu, J., Chen, P., Yang, L., Meng, F., Wang, P.: LogDC: Problem diagnosis for declartively-deployed cloud applications with Log. In: Proceedings of the 2017 IEEE 14th International Conference on E-Business Engineering (ICEBE), pp. 282–287 (2017). https://doi.org/10.1109/ICEBE.2017.52
64. Chen, J., Huang, H., Chen, H.: Informer: irregular traffic detection for containerized microservices RPC in the real world. In: Proceedings of the 4th ACM/IEEE Symposium on Edge Computing. SEC '19, pp. 389–394. Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3318216.3363375
65. Pan, Y., Ma, M., Jiang, X., Wang, P.: Faster, deeper, easier: crowdsourcing diagnosis of microservice kernel failure from user space. In: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis. ISSTA 2021, pp. 646–657. Association for Computing Machinery, New York, NY, USA (2021). https://doi.org/10.1145/3460319.3464805
66. Tien, C.-W., Huang, T.-Y., Tien, C.-W., Huang, T.-C., Kuo, S.-Y.: KubAnomaly: Anomaly detection for the Docker orchestration platform with neural network approaches. Engineering Reports **1**(5), 12080 (2019). https://doi.org/10.1002/eng2.12080
67. Du, M., Li, F., Zheng, G., Srikumar, V.: DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. CCS '17, pp. 1285–1298. Association for Computing Machinery, New York, NY, USA (2017). https://doi.org/10.1145/3133956.3134015
68. Wang, L., Zhao, N., Chen, J., Li, P., Zhang, W., Sui, K.: Root-Cause metric location for microservice systems via log anomaly detection. In: Proceedings of the 2020 IEEE International Conference on Web Services (ICWS), pp. 142–150 (2020). https://doi.org/10.1109/ICWS49710.2020.00026
69. Neves, F., Machado, N., Vilaça, R., Pereira, J.: Horus: Non-intrusive causal analysis of distributed systems logs. In: Proceedings of the 2021 51st Annual IEEE/IFIP International Conference

on Dependable Systems And Networks (DSN), pp. 212–223 (2021). https://doi.org/10.1109/DSN48987.2021.00035. ISSN: 2158-3927
70. Sanvito, D., Siracusano, G., Santhanam, S., Gonzalez, R., Bifulco, R.: syslrn: learning what to monitor for efficient anomaly detection. In: Proceedings of the 2nd European Workshop on Machine Learning And Systems. EuroMLSys '22, pp. 64–71. Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3517207.3526979
71. Bento, A., Correia, J., Filipe, R., Araujo, F., Cardoso, J.: Automated analysis of distributed tracing: Challenges and research directions. Journal of Grid Computing **19**(1), 9 (2021). https://doi.org/10.1007/s10723-021-09551-5
72. Huang, Z., Chen, P., Yu, G., Chen, H., Zheng, Z.: Sieve: Attention-based sampling of End-to-End trace data in distributed microservice systems. In: Proceedings of the 2021 IEEE International Conference on Web Services (ICWS), pp. 436–446 (2021). https://doi.org/10.1109/ICWS53863.2021.00063
73. Guo, X., Peng, X., Wang, H., Li, W., Jiang, H., Ding, D., Xie, T., Su, L.: Graph-based trace analysis for microservice architecture understanding and problem diagnosis. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE 2020, pp. 1387–1397. Association for Computing Machinery, New York, NY, USA (2020). https://doi.org/10.1145/3368089.3417066
74. Li, Z., Chen, J., Jiao, R., Zhao, N., Wang, Z., Zhang, S., Wu, Y., Jiang, L., Yan, L., Wang, Z., Chen, Z., Zhang, W., Nie, X., Sui, K., Pei, D.: Practical root cause localization for microservice systems via trace analysis. In: Proceedings of the 2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS), pp. 1–10 (2021). https://doi.org/10.1109/IWQOS52092.2021.9521340. ISSN: 1548-615X
75. Las-Casas, P., Papakerashvili, G., Anand, V., Mace, J.: Sifter: Scalable sampling for distributed traces, without feature engineering. In: Proceedings of the ACM Symposium on Cloud Computing. SoCC '19, pp. 312–324. Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3357223.3362736
76. Zhang, S., Pan, Z., Liu, H., Jin, P., Sun, Y., Ouyang, Q., Wang, J., Jia, X., Zhang, Y., Yang, H., Zou, Y., Pei, D.: Efficient and robust trace anomaly detection for large-scale microservice systems. In: Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), pp. 69–79. https://doi.org/10.1109/ISSRE59848.2023.00012 . ISSN: 2332-6549
77. Ma, S.-P., Fan, C.-Y., Chuang, Y., Lee, W.-T., Lee, S.-J., Hsueh, N.-L.: Using service dependency graph to analyze and test microservices. In: Proceedings of the 2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC), vol. 02, pp. 81–86 (2018). https://doi.org/10.1109/COMPSAC.2018.10207. ISSN: 0730-3157
78. Liu, P., Xu, H., Ouyang, Q., Jiao, R., Chen, Z., Zhang, S., Yang, J., Mo, L., Zeng, J., Xue, W., Pei, D.: Unsupervised detection of microservice trace anomalies through service-level deep bayesian networks. In: Proceedings of the 2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE), pp. 48–58 (2020). https://doi.org/10.1109/ISSRE5003.2020.00014. ISSN: 2332-6549
79. Cai, Y., Han, B., Su, J., Wang, X.: TraceModel: An automatic anomaly detection and root cause localization framework for microservice systems. In: Proceedings of the 2021 17th International Conference on Mobility, Sensing and Networking (MSN), pp. 512–519 (2021). https://doi.org/10.1109/MSN53354.2021.00081
80. Yu, G., Chen, P., Chen, H., Guan, Z., Huang, Z., Jing, L., Weng, T., Sun, X., Li, X.: MicroRank: End-to-End latency issue localization with extended spectrum analysis in microservice environments. In: Proceedings of the Web Conference 2021. WWW '21, pp. 3087–3098. Association for Computing Machinery, New York, NY, USA (2021). https://doi.org/10.1145/3442381.3449905
81. Ye, Z., Chen, P., Yu, G.: T-Rank:A Lightweight Spectrum based Fault Localization Approach for Microservice Systems. In: Proceedings of the 2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid), pp. 416–425 (2021). https://doi.org/10.1109/CCGrid51090.2021.00051
82. Yu, G., Huang, Z., Chen, P.: TraceRank: Abnormal service localization with dis-aggregated end-to-end tracing data in cloud native systems. Journal of Software: Evolution and Process **n/a**(n/a), 2413 (2021). https://doi.org/10.1002/smr.2413
83. Zhang, K., Zhang, C., Peng, X., Sha, C.: Putracead: Trace anomaly detection with partial labels based on gnn and pu learning. In: Proceedings of the 2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE), pp. 239–250 (2022). https://doi.org/10.1109/ISSRE55969.2022.00032
84. Huang, L., Zhu, T.: tprof: Performance profiling via structural aggregation and automated analysis of distributed systems traces. In: Proceedings of the ACM Symposium on Cloud Computing, pp. 76–91. Association for Computing Machinery, New York, NY, USA (2021)
85. Bogatinovski, J., Nedelkoski, S., Cardoso, J., Kao, O.: Self-Supervised Anomaly Detection from Distributed Traces. In: Proceedings of the 2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC), pp. 342–347 (2020). https://doi.org/10.1109/UCC48980.2020.00054
86. Cao, C., Blaise, A., Verwer, S., Rebecchi, F.: Learning State Machines to Monitor and Detect Anomalies on a Kubernetes Cluster. In: Proceedings of the 17th International Conference on Availability, Reliability And Security. ARES '22, pp. 1–9. Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3538969.3543810
87. Zuo, Y., Wu, Y., Min, G., Huang, C., Pei, K.: An Intelligent Anomaly Detection Scheme for Micro-Services Architectures With Temporal and Spatial Data Analysis. IEEE Transactions on Cognitive Communications and Networking **6**(2), 548–561 (2020). https://doi.org/10.1109/TCCN.2020.2966615
88. Jacob, S., Qiao, Y., Lee, B.: Detecting Cyber Security Attacks Against a Microservices Application Using Distributed Tracing. In: Proceedings of the 7th International Conference on Information Systems Security and Privacy - ICISSP, pp. 588–595. SciTePress, ??? (2021). https://doi.org/10.5220/0010308905880595 . INSTICC
89. Wang, T., Zhang, W., Xu, J., Gu, Z.: Workflow-Aware Automatic Fault Diagnosis for Microservice-Based Applications With Statistics. IEEE Trans. Netw. Serv. Manage. **17**(4), 2350–2363 (2020). https://doi.org/10.1109/TNSM.2020.3022028
90. Chen, H., Wei, K., Li, A., Wang, T., Zhang, W.: Trace-based intelligent fault diagnosis for microservices with deep learning. In: Proceedings of the 2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC), pp. 884–893 (2021). https://doi.org/10.1109/COMPSAC51774.2021.00121. ISSN: 0730-3157
91. Pahl, M.-O., Aubet, F.-X.: All Eyes on You: Distributed Multi-Dimensional IoT Microservice Anomaly Detection. In: Proceedings of the 2018 14th International Conference on Network and Service Management (CNSM), pp. 72–80 (2018). ISSN: 2165-963X
92. Ma, M., Lin, W., Pan, D., Wang, P.: ServiceRank: Root Cause Identification of Anomaly in Large-Scale Microservice Architecture. IEEE Transactions on Dependable and Secure Computing, 1–1 (2021) https://doi.org/10.1109/TDSC.2021.3083671
93. Ravichandiran, R., Bannazadeh, H., Leon-Garcia, A.: Anomaly Detection using Resource Behaviour Analysis for Autoscaling

systems. In: Proceedings of the 2018 4th IEEE Conference on Network Softwarization and Workshops (NetSoft), pp. 192–196 (2018). https://doi.org/10.1109/NETSOFT.2018.8460025
94. Jin, M., Lv, A., Zhu, Y., Wen, Z., Zhong, Y., Zhao, Z., Wu, J., Li, H., He, H., Chen, F.: An Anomaly Detection Algorithm for Microservice Architecture Based on Robust Principal Component Analysis. IEEE Access **8**, 226397–226408 (2020). https://doi.org/10.1109/ACCESS.2020.3044610
95. Baye, G., Hussain, F., Oracevic, A., Hussain, R., Ahsan Kazmi, S.M.: API Security in Large Enterprises: Leveraging Machine Learning for Anomaly Detection. In: Proceedings of the 2021 International Symposium on Networks, Computers and Communications (ISNCC), pp. 1–6 (2021). https://doi.org/10.1109/ISNCC52172.2021.9615638
96. Ma, M., Xu, J., Wang, Y., Chen, P., Zhang, Z., Wang, P.: AutoMAP: Diagnose Your Microservice-based Web Applications Automatically. In: Proceedings of The Web Conference 2020. WWW '20, pp. 246–258. Association for Computing Machinery, New York, NY, USA (2020). https://doi.org/10.1145/3366423.3380111
97. Wu, L., Tordsson, J., Bogatinovski, J., Elmroth, E., Kao, O.: MicroDiag: Fine-grained Performance Diagnosis for Microservice Systems. In: Proceedings of the 2021 IEEE/ACM International Workshop on Cloud Intelligence (CloudIntelligence), pp. 31–36 (2021). https://doi.org/10.1109/CloudIntelligence52565.2021.00015
98. Wu, L., Tordsson, J., Elmroth, E., Kao, O.: MicroRCA: Root Cause Localization of Performance Issues in Microservices. In: Proceedings of the NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium, pp. 1–9 (2020). https://doi.org/10.1109/NOMS47738.2020.9110353 . ISSN: 2374-9709
99. Mukherjee, J., Baluta, A., Litoiu, M., Krishnamurthy, D.: RAD: Detecting Performance Anomalies in Cloud-based Web Services. In: Proceedings of the 2020 IEEE 13th International Conference on Cloud Computing (CLOUD), pp. 493–501 (2020). https://doi.org/10.1109/CLOUD49709.2020.00073 . ISSN: 2159-6190
100. Fourati, M.H., Marzouk, S., Drira, K., Jmaiel, M.: DOCKER-ANALYZER : Towards Fine Grained Resource Elasticity for Microservices-Based Applications Deployed with Docker. In: Proceedings of the 2019 20th International Conference on Parallel and Distributed Computing, Applications and Technologies (PDCAT), pp. 220–225 (2019). https://doi.org/10.1109/PDCAT46702.2019.00049 . ISSN: 2640-6721
101. Zang, X., Chen, W., Zou, J., Zhou, S., Lisong, H., Ruigang, L.: A Fault Diagnosis Method for Microservices Based on Multi-Factor Self-Adaptive Heartbeat Detection Algorithm. In: Proceedings of the 2018 2nd IEEE Conference on Energy Internet and Energy System Integration (EI2), pp. 1–6 (2018). https://doi.org/10.1109/EI2.2018.8582217
102. Shan, H., Chen, Y., Liu, H., Zhang, Y., Xiao, X., He, X., Li, M., Ding, W.: ?-Diagnosis: Unsupervised and Real-time Diagnosis of Small- window Long-tail Latency in Large-scale Microservice Platforms. In: Proceedings of The World Wide Web Conference. WWW '19, pp. 3215–3222. Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3308558.3313653
103. Zhang, Z., Li, B., Wang, J., Liu, Y.: Aamr: Automated anomalous microservice ranking in cloud-native environment. In: Proceedings of the Thirty Third International Conference on Software Engineering and Knowledge Engineering (SEKE 2021), vol. 2021-July, pp. 86–91 (2021). https://doi.org/10.18293/SEKE2021-091 . ISSN: 2325-9000
104. Chen, Z., Liu, J., Su, Y., Zhang, H., Ling, X., Yang, Y., Lyu, M.R.: Adaptive Performance Anomaly Detection for Online Service Systems via Pattern Sketching. In: Proceedings of the 44th International Conference on Software Engineering, pp. 61–72 (2022). https://doi.org/10.1145/3510003.3510085
105. Huang, T., Chen, P., Li, R.: A Semi-Supervised VAE Based Active Anomaly Detection Framework in Multivariate Time Series for Online Systems. In: Proceedings of the ACM Web Conference 2022. WWW '22, pp. 1797–1806. Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3485447.3511984
106. Samir, A., Pahl, C.: DLA: Detecting and Localizing Anomalies in Containerized Microservice Architectures Using Markov Models. In: Proceedings of the 2019 7th International Conference on Future Internet of Things and Cloud (FiCloud), pp. 205–213 (2019). https://doi.org/10.1109/FiCloud.2019.00036
107. Zhang, S., Jin, P., Lin, Z., Sun, Y., Zhang, B., Xia, S., Li, Z., Zhong, Z., Ma, M., Jin, W., Zhang, D., Zhu, Z., Pei, D.: Robust failure diagnosis of microservice system through multimodal data. IEEE Transactions on Services Computing **16**(6), 3851–3864 (2023). https://doi.org/10.1109/TSC.2023.3290018
108. Ren, R., Wang, Y., Liu, F., Li, Z., Xie, G.: Triple:the interpretable deep learning anomaly detection framework based on trace-metric-log of microservice. In: Proceedings of the 2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS), pp. 1–10. https://doi.org/10.1109/IWQoS57198.2023.10188773 . ISSN: 2766-8568
109. Huang, J., Yang, Y., Yu, H., Li, J., Zheng, X.: Twin graph-based anomaly detection via attentive multi-modal learning for microservice system. In: Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 66–78. https://doi.org/10.1109/ASE56229.2023.00138 . ISSN: 2643-1572
110. Silva, M.d., Daniel, S., Kumarapeli, M., Mahadura, S., Rupasinghe, L., Liyanapathirana, C.: Anomaly Detection in Microservice Systems Using Autoencoders. In: Proceedings of the 2022 4th International Conference on Advancements in Computing (ICAC), pp. 488–493 (2022). https://doi.org/10.1109/ICAC57685.2022.10025259
111. Ghorbani, M., Moghaddam, F.F., Zhang, M., Pourzandi, M., Nguyen, K.K., Cheriet, M.: DistAppGaurd: Distributed Application Behaviour Profiling in Cloud-Based Environment. In: Proceedings of the Annual Computer Security Applications Conference. ACSAC, pp. 837–848. Association for Computing Machinery, New York, NY, USA (2021). https://doi.org/10.1145/3485832.3485907
112. Shahini, S., Momeni, H.: Autoencoder-based anomaly detection in microservices using distributed tracing. In: Proceedings of the 2024 20th CSI International Symposium on Artificial Intelligence and Signal Processing (AISP), pp. 1–6. https://doi.org/10.1109/AISP61396.2024.10475273 . ISSN: 2640-5768
113. Liu, H., Huang, X., Jia, M., Jia, T., Han, J., Li, Y., Wu, Z.: UAC-AD: Unsupervised adversarial contrastive learning for anomaly detection on multi-modal data in microservice systems. IEEE Transactions on Services Computing (2024). https://doi.org/10.1109/TSC.2024.3411481
114. Tan, R., Li, Z.: MAAD: A distributed anomaly detection architecture for microservices systems. In: Proceedings of the 2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 1009–1021. https://doi.org/10.1109/IPDPS57955.2024.00094 . ISSN: 1530-2075
115. Zhou, S., Zhan, X., Li, L., Liu, Y.: Effective anomaly detection for microservice systems with real-time feature selection. In: Proceedings of the 2023 30th Asia-Pacific Software Engineering Conference (APSEC), pp. 101–110. https://doi.org/10.1109/APSEC60848.2023.00020 . ISSN: 2640-0715
116. Duan, C., Jia, T., Li, Y., Huang, G.: AcLog: An approach to detecting anomalies from system logs with active learning. In: Proceedings of the 2023 IEEE International Conference on Web

Services (ICWS), pp. 436–443. https://doi.org/10.1109/ICWS60048.2023.00062 . ISSN: 2836-3868
117. Duan, C., Jia, T., Cai, H., Li, Y., Huang, G.: AFALog: A general augmentation framework for log-based anomaly detection with active learning. In: Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), pp. 46–56. https://doi.org/10.1109/ISSRE59848.2023.00068 . ISSN: 2332-6549
118. Chen, H., Chen, P., Yu, G.: A Framework of Virtual War Room and Matrix Sketch-Based Streaming Anomaly Detection for Microservice Systems. IEEE Access **8**, 43413–43426 (2020). https://doi.org/10.1109/ACCESS.2020.2977464
119. Luo, P., Qiu, C., Wang, Y.: DAS: Deep Autoencoder with Scoring Neural Network for Anomaly Detection. In: Proceedings of the 2020 3rd International Conference on Algorithms, Computing and Artificial Intelligence. ACAI 2020, pp. 1–6. Association for Computing Machinery, New York, NY, USA (2020). https://doi.org/10.1145/3446132.3446181
120. Zaojian, D., Yong, L., Mu, C., Liang, C., Wengao, F., Ziang, L.: Semi-supervised power microservices log anomaly detection based on BiLSTM and BERT with attention. In: Proceedings of the 2023 2nd International Conference on Advanced Electronics, Electrical and Green Energy (AEEGE), pp. 82–87. https://doi.org/10.1109/AEEGE58828.2023.00023
121. Chen, Y., Yan, M., Yang, D., Zhang, X., Wang, Z.: Deep Attentive Anomaly Detection for Microservice Systems with Multimodal Time-Series Data. In: Proceedings of the 2022 IEEE International Conference on Web Services (ICWS), pp. 373–378 (2022). https://doi.org/10.1109/ICWS55610.2022.00062
122. Yang, X., Wang, J., Zhou, B., Wang, W., Liu, W., Dong, Y.: Fine-grained Spatiotemporal Features-Based for Anomaly Detection in Microservice Systems. In: Proceedings of the 2022 IEEE 24th Int Conf on High Performance Computing and Communications; 8th Int Conf on Data Science and Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud and Big Data Systems and Application (HPCC/DSS/SmartCity/DependSys), pp. 847–856 (2022). https://doi.org/10.1109/HPCC-DSS-SmartCity-DependSys57074.2022.00138
123. Raeiszadeh, M., Ebrahimzadeh, A., Saleem, A., Glitho, R.H., Eker, J., Mini, R.A.F.: Real-time anomaly detection using distributed tracing in microservice cloud applications. In: Proceedings of the 2023 IEEE 12th International Conference on Cloud Networking (CloudNet), pp. 36–44. https://doi.org/10.1109/CloudNet59005.2023.10490038 . ISSN: 2771-5663
124. Ding, S., E, Y., Zhang, J., Li, L., Zhang, L., Ge, J.: Trace anomaly detection for microservice systems via graph-based semi-supervised learning. In: Proceedings of the 2024 27th International Conference on Computer Supported Cooperative Work in Design (CSCWD), pp. 2375–2380. https://doi.org/10.1109/CSCWD61410.2024.10580078 . ISSN: 2768-1904
125. Zeng, Z., Zhang, Y., Xu, Y., Ma, M., Qiao, B., Zou, W., Chen, Q., Zhang, M., Zhang, X., Zhang, H., Gao, X., Fan, H., Rajmohan, S., Lin, Q., Zhang, D.: TraceArk: Towards actionable performance anomaly alerting for online service systems. In: Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 258–269. https://doi.org/10.1109/ICSE-SEIP58684.2023.00029 . ISSN: 2832-7659
126. Kalinagac, O., Soussi, W., Anser, Y., Gaber, C., Gür, G.: Root cause and liability analysis in the microservices architecture for edge IoT services. In: Proceedings of the ICC 2023 - IEEE International Conference on Communications, pp. 3277–3283. https://doi.org/10.1109/ICC45041.2023.10279721 . ISSN: 1938-1883
127. Sutton, R.S., Barto, A.G.: Reinforcement Learning, Second Edition: An Introduction. Adaptive Computation and Machine Learning series. MIT Press. https://books.google.pt/books?id=uWV0DwAAQBAJ
128. Belhadi, A., Djenouri, Y., Srivastava, G., Lin, J.-W.: Reinforcement learning multi-agent system for faults diagnosis of mirco-services in industrial settings. Computer Communications **177**, 213–219 (2021). https://doi.org/10.1016/j.comcom.2021.07.010
129. Kim, M., Sumbaly, R., Shah, S.: Root cause detection in a service-oriented architecture. ACM SIGMETRICS Performance Evaluation Review **41**(1), 93–104 (2013). https://doi.org/10.1145/2494232.2465753
130. Lin, W., Ma, M., Pan, D., Wang, P.: FacGraph: Frequent Anomaly Correlation Graph Mining for Root Cause Diagnose in Micro-Service Architecture. In: Proceedings of the 2018 IEEE 37th International Performance Computing and Communications Conference (IPCCC), pp. 1–8 (2018). https://doi.org/10.1109/PCCC.2018.8711092 . ISSN: 2374-9628
131. Gu, S., Rong, G., Ren, T., Zhang, H., Shen, H., Yu, Y., Li, X., Ouyang, J., Chen, C.: TrinityRCL: Multi-Granular and Code-Level Root Cause Localization Using Multiple Types of Telemetry Data in Microservice Systems. IEEE Trans. Software Eng. **49**(5), 3071–3088 (2023). https://doi.org/10.1109/TSE.2023.3241299
132. Yang, J., Guo, Y., Chen, Y., Zhao, Y.: MicroNet: Operation aware root cause identification of microservice system anomalies. IEEE Transactions on Network and Service Management (2024). https://doi.org/10.1109/TNSM.2024.3387552
133. Ma, S.-P., Liu, I.-H., Chen, C.-Y., Lin, J.-T., Hsueh, N.-L.: Version-Based Microservice Analysis, Monitoring, and Visualization. In: Proceedings of the 2019 26th Asia-Pacific Software Engineering Conference (APSEC), pp. 165–172 (2019). https://doi.org/10.1109/APSEC48747.2019.00031 . ISSN: 2640-0715
134. Pan, Y., Ma, M., Jiang, X., Wang, P.: DyCause: Crowdsourcing to Diagnose Microservice Kernel Failure. IEEE Transactions on Dependable and Secure Computing (2023). https://doi.org/10.1109/TDSC.2022.3233915
135. Sun, C.-A., Zeng, T., Zuo, W., Liu, H.: A trace-log-clusterings-based fault localization approach to microservice systems. In: Proceedings of the 2023 IEEE International Conference on Web Services (ICWS), pp. 7–13. https://doi.org/10.1109/ICWS60048.2023.00013 . ISSN: 2836-3868
136. Man, Y., Li, S., Xia, W., Li, Y., Yu, B., Long, Y., Pan, Y.: Detective-dee: A non-intrusive in situ anomaly detection and fault localization framework. In: Proceedings of the 2023 42nd International Symposium on Reliable Distributed Systems (SRDS), pp. 243–253. https://doi.org/10.1109/SRDS60354.2023.00032 . ISSN: 2575-8462
137. Zhou, X., Peng, X., Xie, T., Sun, J., Ji, C., Liu, D., Xiang, Q., He, C.: Latent error prediction and fault localization for microservice applications by learning from system trace logs. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE 2019, pp. 683–694. Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3338906.3338961
138. Afshinpour, B., Groz, R., Amini, M.-R.: Telemetry-Based Software Failure Prediction by Concept-Space Model Creation. In: Proceedings of the 2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS), pp. 199–208 (2022). https://doi.org/10.1109/QRS57517.2022.00030 . ISSN: 2693-9177
139. Kohyarnejadfard, I., Aloise, D., Azhari, S.V., Dagenais, M.R.: Anomaly detection in microservice environments using distributed tracing data analysis and NLP. Journal of Cloud Computing **11**(1), 25 (2022). https://doi.org/10.1186/s13677-022-00296-4
140. Li, G., Wen, Z., Xie, X.: Unsupervised anomaly detection based on cnn-vae with spectral residual for kpis. In: Proceedings of the

2022 IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys), pp. 1307–1313 (2022). https://doi.org/10.1109/HPCC-DSS-SmartCity-DependSys57074.2022.00204

141. Zhang, C., Peng, X., Sha, C., Zhang, K., Fu, Z., Wu, X., Lin, Q., Zhang, D.: DeepTraLog: Trace-Log Combined Microservice Anomaly Detection through Graph-based Deep Learning. In: Proceedings of the 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE), pp. 623–634 (2022). https://doi.org/10.1145/3510003.3510180 . ISSN: 1558-1225
142. He, Z., Chen, P., Li, X., Wang, Y., Yu, G., Chen, C., Li, X., Zheng, Z.: A Spatiotemporal Deep Learning Approach for Unsupervised Anomaly Detection in Cloud Systems. IEEE Transactions on Neural Networks and Learning Systems **34**(4), 1705–1719 (2023). https://doi.org/10.1109/TNNLS.2020.3027736
143. Li, Y., Liu, Y., Wang, H., Chen, Z., Cheng, W., Chen, Y., Yu, W., Chen, H., Liu, C.: GLAD: Content-aware dynamic graphs for log anomaly detection. In: Proceedings of the 2023 IEEE International Conference on Knowledge Graph (ICKG), pp. 9–18. https://doi.org/10.1109/ICKG59574.2023.00007
144. Traini, L., Cortellessa, V.: DeLag: Using Multi-Objective Optimization to Enhance the Detection of Latency Degradation Patterns in Service-Based Systems. IEEE Transactions on Software Engineering, 1–28 (2023) https://doi.org/10.1109/TSE.2023.3266041
145. Xie, Z., Xu, H., Chen, W., Li, W., Jiang, H., Su, L., Wang, H., Pei, D.: Unsupervised Anomaly Detection on Microservice Traces through Graph VAE. In: Proceedings of the ACM Web Conference 2023. WWW '23, pp. 2874–2884. Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3543507.3583215
146. Dodda, S., Chintala, S., Kunchakuri, N., Kamuni, N.: Enhancing Microservice Reliability in Cloud Environments Using Machine Learning for Anomaly Detection. In: 2024 International Conference on Computing, Sciences and Communications (ICCSC), pp. 1–5 (2024). https://doi.org/10.1109/ICCSC62048.2024.10830437
147. He, Y., Zhang, X., Wang, P., Liu, M., Cao, Z.: Microservice traces anomaly detection based on structural clustering and graph attention autoencoder. In: Proceedings of the 2024 7th International Conference on Advanced Algorithms and Control Engineering (ICAACE), pp. 977–981. https://doi.org/10.1109/ICAACE61206.2024.10549242
148. Khudyakov, D.A., Yakhyaeva, G.E.: Unsupervised anomaly detection on distributed log tracing through deep learning. In: Proceedings of the 2024 IEEE 25th International Conference of Young Professionals in Electron Devices and Materials (EDM), pp. 1830–1833. https://doi.org/10.1109/EDM61683.2024.10615125 . ISSN: 2325-419X
149. Liu, X., Liu, Y., Wei, M., Xu, P.: LMGD: Log-Metric Combined Microservice Anomaly Detection Through Graph-Based Deep Learning. IEEE Access **12**, 186510–186519 (2024). https://doi.org/10.1109/ACCESS.2024.3481676
150. Zeng, J., Lai, J., Li, S., Li, X., Yang, R.: Microservice Anomaly Monitoring Method based on Multi-dimensional Spatiotemporal Feature Reconstruction. In: 2024 4th International Symposium on Artificial Intelligence and Intelligent Manufacturing (AIIM), pp. 567–572 (2024). https://doi.org/10.1109/AIIM64537.2024.10934387
151. Zhu, G., Guo, Y., Chen, Y.: MN-GAT: Incorporating Metric Names into Metric Correlation Graphs for Anomaly Detection. In: 2024 7th World Conference on Computing and Communication Technologies (WCCCT), pp. 98–103 (2024). https://doi.org/10.1109/WCCCT60665.2024.10541442
152. Osswald, M., Schönenberger, T., Cantali, G., Soussi, W., Gür, G.: Anomaly Detection in Microservices Architecture Using Graph Neural Networks. In: 2025 33rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing (PDP), pp. 560–567 (2025). https://doi.org/10.1109/PDP66500.2025.00085 . ISSN: 2377-5750
153. Raeiszadeh, M., Ebrahimzadeh, A., Glitho, R.H., Eker, J., Mini, R.A.F.: Asynchronous Real-Time Federated Learning for Anomaly Detection in Microservice Cloud Applications. IEEE Transactions on Machine Learning in Communications and Networking **3**, 176–194 (2025). https://doi.org/10.1109/TMLCN.2025.3527919
154. Bento, A., Correia, J., Duraes, J., Soares, J.a., Ribeiro, L., Ferreira, A., Carreira, R., Araujo, F., Barbosa, R.: A layered framework for root cause diagnosis of microservices. In: Proceedings of the 2021 IEEE 20th International Symposium on Network Computing and Applications (NCA), pp. 1–8 (2021). https://doi.org/10.1109/NCA53618.2021.9685494 . ISSN: 2643-7929
155. Hou, C., Jia, T., Wu, Y., Li, Y., Han, J.: Diagnosing Performance Issues in Microservices with Heterogeneous Data Source. In: Proceedings of the 2021 IEEE Intl Conf on Parallel Distributed Processing with Applications, Big Data Cloud Computing, Sustainable Computing Communications, Social Computing Networking (ISPA/BDCloud/SocialCom/SustainCom), pp. 493–500 (2021). https://doi.org/10.1109/ISPA-BDCloud-SocialCom-SustainCom52081.2021.00074
156. Li, R., Du, M., Wang, Z., Chang, H., Mukherjee, S., Eide, E.: LongTale: Toward Automatic Performance Anomaly Explanation in Microservices. In: Proceedings of the 2022 ACM/SPEC on International Conference on Performance Engineering. ICPE '22, pp. 5–16. Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3489525.3511675
157. Li, M., Li, Z., Yin, K., Nie, X., Zhang, W., Sui, K., Pei, D.: Causal Inference-Based Root Cause Analysis for Online Service Systems with Intervention Recognition. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. KDD '22, pp. 3230–3240. Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3534678.3539041
158. Yang, J., Guo, Y., Chen, Y., Zhao, Y., Lu, Z., Liang, Y.: Robust Anomaly Diagnosis in Heterogeneous Microservices Systems under Variable Invocations. In: Proceedings of the GLOBECOM 2022 - 2022 IEEE Global Communications Conference, pp. 2722–2727 (2022). https://doi.org/10.1109/GLOBECOM48099.2022.10001270
159. Yang, L., Li, J., Shi, K., Yang, S., Yang, Q., Sun, J.: MicroMILTS: Fault Location for Microservices Based Mutual Information and LSTM Autoencoder. In: Proceedings of the 2022 23rd Asia-Pacific Network Operations and Management Symposium (APNOMS), pp. 1–6 (2022). https://doi.org/10.23919/APNOMS56106.2022.9919941 . ISSN: 2576-8565
160. Zhang, Q., Jia, T., Wu, Z., Wu, Q., Jia, L., Li, D., Tao, Y., Xiao, Y.: Fault Localization for Microservice Applications with System Logs and Monitoring Metrics. In: Proceedings of the 2022 7th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA), pp. 149–154 (2022). https://doi.org/10.1109/ICCCBDA55098.2022.9778893
161. Jiang, X., Pan, Y., Ma, M., Wang, P.: Look Deep into the Microservice System Anomaly through Very Sparse Logs. In: Proceedings of the ACM Web Conference 2023. WWW '23, pp. 2970–2978. Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3543507.3583338
162. Kalinagac, O., Soussi, W., Gür, G.: Graph Based Liability Analysis for the Microservice Architecture. In: Proceedings of the 18th International Conference on Network and Service Management.

CNSM '22, pp. 1–3. International Federation for Information Processing, Laxenburg, AUT (2023)
163. Li, Y., Lu, Y., Wang, J., Qi, Q., Wang, J., Wang, Y., Liao, J.: TADL: Fault Localization with Transformer-based Anomaly Detection for Dynamic Microservice Systems. In: Proceedings of the 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 718–722 (2023). https://doi.org/10.1109/SANER56733.2023.00078 . ISSN: 2640-7574
164. Xin, R., Chen, P., Zhao, Z.: CausalRCA: Causal inference based precise fine-grained root cause localization for microservice applications. Journal of Systems and Software **203**, 111724 (2023). https://doi.org/10.1016/j.jss.2023.111724
165. Borse, H., Satapathy, U., Mandal, M., Mitra, B.: URCD: Unsupervised Root Cause Detection in Microservices Architecture with HGAN. In: 2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS), pp. 1423–1426 (2024). https://doi.org/10.1109/ICDCS60910.2024.00137 . ISSN: 2575-8411
166. Chen, Y., Xiao, Z.-M., Teng, F.: A Root Cause Localization Method Based on Event Call Chains for Microservices. In: 2024 16th International Conference on Communication Software and Networks (ICCSN), pp. 43–48 (2024). https://doi.org/10.1109/ICCSN63464.2024.10793332 . ISSN: 2472-8489
167. Chen, R., Peng, F., Ji, X., Xiang, N., Lou, Y., Zhang, K., Pu, Y., Wu, W.: Graph-Based Ensemble Learning for Enhanced Fault Localization in Microservices. In: 2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC), pp. 3356–3362 (2024). https://doi.org/10.1109/SMC54092.2024.10831257
168. Cheng, H., Li, Q., Liu, B., Liu, S., Pan, L.: DGERCL: A Dynamic Graph Embedding Approach for Root Cause Localization in Microservice Systems. IEEE Trans. Serv. Comput **17**(6), 3417–3428 (2024). https://doi.org/10.1109/TSC.2024.3437742
169. Enz, S., Nigg, S., Kalinagac, O., Ermis, O., Gür, G.: ML driven root cause analysis (RCA) in telco microservices continuum. In: Proceedings of the 2024 IEEE International Conference on Communications Workshops (ICC Workshops), pp. 371–377. https://doi.org/10.1109/ICCWorkshops59551.2024.10615702 . ISSN: 2694-2941
170. Gao, H., Zhao, J., Li, W., Li, Z.: MicroMCM: Fine-grained Root Cause Localization for Microservice Systems Based on Multiple Causal Inference Methods. In: 2024 27th International Conference on Computer Supported Cooperative Work in Design (CSCWD), pp. 371–376 (2024). https://doi.org/10.1109/CSCWD61410.2024.10580445 . ISSN: 2768-1904
171. Han, Y., Du, Q., Huang, Y., Li, P., Shi, X., Wu, J., Fang, P., Tian, F., He, C.: Holistic root cause analysis for failures in cloud-native systems through observability data. IEEE Trans. Serv. Comput. **17**(6), 3789–3802 (2024). https://doi.org/10.1109/TSC.2024.3478759
172. Han, Y., Du, Q., Huang, Y., Wu, J., Tian, F., He, C.: The potential of one-shot failure root cause analysis: Collaboration of the large language model and small classifier. In: 2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 931–943 (2024). ISSN: 2643-1572
173. Huang, Y., Du, Q., Han, Y., He, C., Tian, F.: Semi-supervised metrics-based self-training root cause analysis for cloud-native systems with class-imbalanced data. In: ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 6405–6409 (2024). https://doi.org/10.1109/ICASSP48485.2024.10447959. ISSN: 2379-190X
174. Huang, Y., Zhang, J., Chai, X., Sun, Y.: TopoRCA: A lightweight root cause analysis system based on application topology. In: 2024 27th International Conference on Computer Supported Cooperative Work in Design (CSCWD), pp. 2943–2948 (2024). https://doi.org/10.1109/CSCWD61410.2024.10580442. ISSN: 2768-1904
175. Li, P., Du, Q., Zhao, S., Fang, P.: Advancing root cause analysis in cloud-native system with knowledge graph path embedding translation. In: 2024 27th International Conference on Computer Supported Cooperative Work in Design (CSCWD), pp. 3134–3139 (2024). https://doi.org/10.1109/CSCWD61410.2024.10580547. ISSN: 2768-1904
176. Pham, L., Ha, H., Zhang, H.: Root cause analysis for microservices based on causal inference: How far are we? In: 2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 706–718 (2024). ISSN: 2643-1572
177. Ren, R., Yang, J., Yang, L., Gu, X., Sun, L.: SLIM: a scalable and interpretable light-weight fault localization algorithm for imbalanced data in microservice. In: 2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 27–39 (2024). ISSN: 2643-1572
178. Sun, Y., Shi, B., Mao, M., Ma, M., Xia, S., Zhang, S., Pei, D.: ART: A unified unsupervised framework for incident management in microservice systems. In: 2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1183–1194 (2024). ISSN: 2643-1572
179. Tao, L., Lu, X., Zhang, S., Luan, J., Li, Y., Li, M., Li, Z., Yu, Q., Xie, H., Xu, R., Hu, C., Yang, C., Pei, D.: Diagnosing performance sssues for large-scale microservice systems With heterogeneous Graph. IEEE Trans. Serv. Comput **17**(5), 2223–2235 (2024). https://doi.org/10.1109/TSC.2024.3402172
180. Wang, X., Liu, X., Xu, P., Du, H.: Graph-based anomaly detection and root cause analysis for microservices in Cloud-native platform. In: 2024 IEEE International Conference on Sustainable Computing and Communications (SustainCom), pp. 1–7 (2024). https://doi.org/10.1109/SustainCom63171.2024.00009
181. Wang, Y., Zhu, Z., Fu, Q., Ma, Y., He, P.: MRCA: Metric-level root cause analysis for microservices via Multi-modal data. In: 2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1057–1068 (2024). ISSN: 2643-1572
182. Wang, J., Li, Y., Qi, Q., Lu, Y., Wu, B.: Multilayered fault detection and localization with transformer for microservice systems. IEEE Transactions on Reliability (2024). https://doi.org/10.1109/TR.2024.3356717
183. Wu, Z., Wang, J., Qi, Q., Shu, M.-G., Chu, R., Li, J.-B., Jin, J., Chen, D.: FlowRCA: enhancing microservice reliability with Non-invasive root cause analysis. In: 2024 IEEE International Conference on Web Services (ICWS), pp. 1251–1258 (2024). https://doi.org/10.1109/ICWS62655.2024.00149. ISSN: 2836-3868
184. Xu, Y., Qiu, Z., Gao, H., Zhao, X., Wang, L., Li, R.: Heterogeneous data-driven failure diagnosis for microservice-based industrial clouds toward consumer digital ecosystems. IEEE Transactions on Consumer Electronics **70**(1), 2027–2037 (2024). https://doi.org/10.1109/TCE.2023.3337351
185. Yao, Z., Ye, H., Pei, C., Cheng, G., Wang, G., Liu, Z., Chen, H., Cui, H., Li, Z., Li, J., Xie, G., Pei, D.: SparseRCA: Unsupervised root cause analysis in sparse microservice testing traces. In: 2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE), pp. 391–402 (2024). https://doi.org/10.1109/ISSRE62328.2024.00045. ISSN: 2332-6549
186. Zhang, L., Shi, Y., Qi, K., Wu, D., Wang, X., Yan, Z., Chen, Z.: MicroHFRCL: A history faults based root cause localization framework in microservice systems. In: 2024 International Joint Conference on Neural Networks (IJCNN), pp. 1–8 (2024). https://doi.org/10.1109/IJCNN60899.2024.10650929. ISSN: 2161-4407
187. Zhang, C., Dong, Z., Peng, X., Zhang, B., Chen, M.: Trace-based multi-dimensional root cause localization of performance issues in microservice systems. In: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE), pp. 1347–1358. ISSN: 1558-1225. https://ieeexplore.ieee.org/document/10549362 Accessed 2024-08-15

188. Wu, L., Tordsson, J., Acker, A., Kao, O.: MicroRAS: Automatic recovery in the absence of historical failure data for microservice systems. In: Proceedings of the 2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC), pp. 227–236 (2020). https://doi.org/10.1109/UCC48980.2020.00041
189. Wang, P., Xu, J., Ma, M., Lin, W., Pan, D., Wang, Y., Chen, P.: CloudRanger: root cause identification for cloud native systems. In: Proceedings of the 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing. CCGrid '18, pp. 492–502. IEEE Press, Washington, District of Columbia (2018). https://doi.org/10.1109/CCGRID.2018.00076
190. Ji, S., Wu, W., Pu, Y.: Multi-indicators prediction in microservice using Granger causality test and Attention LSTM. In: Proceedings of the 2020 IEEE World Congress on Services (SERVICES), pp. 77–82 (2020). https://doi.org/10.1109/SERVICES48979.2020.00030 . ISSN: 2642-939X
191. Cai, Y., Han, B., Li, J., Zhao, N., Su, J.: ModelCoder: A fault model based automatic root cause localization framework for microservice systems. In: Proceedings of the 2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS), pp. 1–6 (2021). https://doi.org/10.1109/IWQOS52092.2021.9521318. ISSN: 1548-615X
192. Sun, Y., Zhao, L., Wang, Z., Cui, D., Yang, Y., Gao, Z.: Fault Root Rank Algorithm Based on Random Walk Mechanism in Fault Knowledge Graph. In: Proceedings of the 2021 IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB), pp. 1–6 (2021). https://doi.org/10.1109/BMSB53066.2021.9547194 . ISSN: 2155-5052
193. Xie, R., Yang, J., Li, J., Wang, L.: ImpactTracer: Root cause localization in microservices based on fault propagation modeling. In: Proceedings of the 2023 Design, Automation and Test in Europe Conference and Exhibition (DATE), pp. 1–6. https://doi.org/10.23919/DATE56975.2023.10137078 . ISSN: 1558-1101
194. Yao, X.-W., Lu, Q.-C., Li, Q., Liu, L.-L., Zhu, Z.-C.: MicroKGCL: A knowledge graph for root cause localization of feedback issues in microservices. In: Proceedings of the 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS), pp. 628–637. https://doi.org/10.1109/QRS60937.2023.00067 . ISSN: 2693-9177
195. Jeh, G., Widom, J.: Scaling personalized web search. In: Proceedings of the 12th International Conference on World Wide Web. WWW '03, pp. 271–279. Association for Computing Machinery, New York, NY, USA (2003). https://doi.org/10.1145/775152.775191
196. BLUEPERF: Acme Air Main Service - Java/Liberty. https://github.com/blueperf/acmeair-mainservice-java Accessed 2023-06-21
197. Authors, I.: BookInfo. https://istio-releases.github.io/v0.1/docs/samples/bookinfo.html Accessed 2023-06-20
198. Cornell University, S.: Microservices Death Star. http://microservices.ece.cornell.edu/ Accessed 2023-06-20
199. L'Aquila, S..U.: E-shopper. https://github.com/SEALABQualityGroup/E-Shopper Accessed 2023-06-21
200. Balouek, D., Amarie, A.C., Charrier, G., Desprez, F., Jeannot, E., Jeanvoine, E., Lèbre, A., Margery, D., Niclausse, N., Nussbaum, L., Richard, O., Perez, C., Quesnel, F., Rohr, C., Sarzyniec, L.: Adding Virtualization Capabilities to the Grid'5000 Testbed. In: Ivanov, I.I., Sinderen, M., Leymann, F., Shan, T. (eds.) Proceedings of the Cloud Computing and Services Science. Communications in Computer and Information Science, pp. 3–20. Springer, Cham (2013). https://doi.org/10.1007/978-3-319-04519-1_1
201. Lightstep: Hipster Shop: Cloud-Native Observability Demo Application. https://github.com/lightstep/hipster-shop Accessed 2023-06-20
202. Google: Sample cloud-first application with 10 microservices showcasing Kubernetes, Istio, and gRPC. https://github.com/GoogleCloudPlatform/microservices-demo Accessed 2023-06-21
203. Shrivastwa, A., Sarat, S., Jackson, K., Bunch, C., Sigler, E., Campbell, T.: OpenStack: Building a Cloud Environment. Packt Publishing. https://books.google.pt/books?id=uYMNvgAACAAJ
204. Rajagopalan, S.: pymicro. https://github.com/rshriram/pymicro Accessed 2023-06-20
205. Weaveworks, I.: Microservices Demo: Sock Shop. https://microservices-demo.github.io/ Accessed 2023-06-20
206. Instana: Sample Microservice Application. https://github.com/instana/robot-shop Accessed 2023-06-21
207. TPC: TPC-W Homepage. https://tpc.org/tpcw/ Accessed 2023-06-21
208. Fudan University, S.E.L.: Train Ticket - A Benchmark Microservice System. https://github.com/FudanSELab/train-ticket Accessed 2021-11-15
209. NetManAIOps: AIOps-Challenge-2020-Data. https://github.com/NetManAIOps/AIOps-Challenge-2020-Data Accessed 2023-06-20
210. Cao, C., Blaise, A.: AssureMOSS Kubernetes Run-time Monitoring Dataset. 4TU.ResearchData.https://doi.org/10.4121/20463687
211. LOGPAI: Loghub. https://github.com/logpai/loghub/blob/1629e497268d1e727fcdc9c1412aa321441added/BGL/README.md Accessed 2023-06-22
212. Cloudwise: GAIA - Generic AIOps Atlas. https://docs.aiops.cloudwise.com/en/gaia/overview.html Accessed 2024-08-19
213. Kaggle: loghub-logbert-hdfs-dataset. https://kaggle.com/code/nowlookahead/loghub-logbert-hdfs-dataset Accessed 2024-08-18
214. Huang, Z.: a18499/KubAnomaly_DataSet. https://github.com/a18499/KubAnomaly_DataSet Accessed 2023-06-22
215. Smith, B.: Introducing the new Serverless LAMP stack | AWS Compute Blog. Section: Amazon API Gateway. https://aws.amazon.com/pt/blogs/compute/introducing-the-new-serverless-lamp-stack/ Accessed 2023-06-22
216. Jiang, X.: MicroCU. https://github.com/jxrjxrjxr/MicroCU Accessed 2023-06-22
217. Nedelkoski, S., Bogatinovski, J., Mandapati, A.K., Becker, S., Cardoso, J., Kao, O.: Multi-Source Distributed System Data for AI-powered Analytics. Zenodo (2019). https://doi.org/10.5281/zenodo.3549604. (**Accessed 2024-08-25**)
218. NetflixOSS: Netflix Open Source Software Center. https://netflix.github.io/ Accessed 2023-06-22
219. OpenStack: A large collection of system log datasets for AI-driven log analytics [ISSRE'23] - logpai/loghub. https://github.com/logpai/loghub/blob/master/OpenStack/README.md Accessed 2024-08-18
220. Lab., T.N.: OmniAnomaly/ServerMachineDataset at master NetManAIOps/OmniAnomaly. https://github.com/NetManAIOps/OmniAnomaly Accessed 2023-06-22
221. Sanvito, D., Siracusano, G., Santhanam, S., Gonzalez, R., Bifulco, R.: syslrn: Learning What to Monitor for Efficient Anomaly Detection [Dataset]. NEC Laboratories Europe. https://github.com/nec-research/syslrn-EuroMLSys22 Accessed 2023-06-22
222. NetManAIOps: TraceRCA. https://github.com/NetManAIOps/TraceRCA Accessed 2023-06-22
223. Yahoo, L.: Webscope | Yahoo Labs. https://webscope.sandbox.yahoo.com/catalog.php?datatype=s&did=70 Accessed 2023-06-21
224. ScalingLab: ScalingLab/AutoLog. https://github.com/ScalingLab/AutoLog Accessed 2023-06-22
225. NetManAIOps: CIRCA - Causal Inference-based Root Cause Analysis. https://github.com/NetManAIOps/CIRCA Accessed 2023-06-22
226. AXinx: CausalRCA_code. https://github.com/AXinx/CausalRCA_code Accessed 2023-06-22
227. Joseph, C.T., Chandrasekaran, K.: Straddling the crevasse: A review of microservice software architecture foundations and

recent advancements. Software: Practice and Experience **49**(10), 1448–1484 (2019). https://doi.org/10.1002/spe.2729
228. Sabt, M., Achemlal, M., Bouabdallah, A.: Trusted execution environment: What it is, and what it is not. In: 2015 IEEE Trustcom/BigDataSE/ISPA, vol. 1, pp. 57–64 (2015). https://doi.org/10.1109/Trustcom.2015.357
229. Zheng, W., Wu, Y., Wu, X., Feng, C., Sui, Y., Luo, X., Zhou, Y.: A survey of intel SGX and its applications. Frontiers of Computer Science **15**(3), 153808 (2021). https://doi.org/10.1007/s11704-019-9096-y
230. Muñoz, A., Ríos, R., Román, R., López, J.: A survey on the (in) security of trusted execution environments. Computers & Security **129**, 103180 (2023). https://doi.org/10.1016/j.cose.2023.103180
231. Arnautov, S., Trach, B., Gregor, F., Knauth, T., Martin, A., Priebe, C., Lind, J., Muthukumaran, D., O'Keeffe, D., Stillwell, M.: Scone: Secure linux containers with intel sgx. In: Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), pp. 689-703 (2016). https://www.usenix.org/conference/osdi16/technical-sessions/presentation/arnautov
232. Hunt, T., Zhu, Z., Xu, Y., Peter, S., Witchel, E.: Ryoan: A distributed sandbox for untrusted computation on secret data. ACM Transactions on Computer Systems **35**(4) (2018) https://doi.org/10.1145/3183440
233. Harnik, D., Margalit, O., Naor, M., Tsfadia, A.: Confidential logging using trusted execution environments. In: Proceedings of the 28th USENIX Security Symposium, pp. 1627–1644 (2019)
234. Hahnle, E., Henze, M., Hummen, R., Wehrle, K.: Anomaly detection with trusted execution: Privacy-preserving systems for edge clouds. Computer Communications **160**, 415–429 (2020). https://doi.org/10.1016/j.comcom.2020.07.020
235. Li, T., Sahu, A.K., Talwalkar, A., Smith, V.: Federated learning: Challenges, methods, and future directions. IEEE Signal Process. Mag. **37**(3), 50–60 (2020). https://doi.org/10.1109/MSP.2020.2975749
236. Hardy, S., Henecka, W., Ivey-Law, H., Nock,R., Patrini, G., Smith, B., Thorne, G.: Private federated learning on vertically partitioned data using tees. arXiv preprint arXiv:1711.10677 (2017)
237. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. IEEE Inter. Things J. **3**(5), 637–646 (2016). https://doi.org/10.1109/JIOT.2016.2579198
238. Liu, B., Ding, M., Yu, H., Wang, Z., Chen, Y .: Adaptive federated learning in resource-constrained edge computing systems. IEEE J. Sel. Areas Commun. **39**(1), 120–135 (2020). https://doi.org/10.1109/JSAC.2020.3036964
239. Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, B., Patel, S., Ramage, D., Segal, A., Seth, K.: Practical secure aggregation for privacy-preserving machine learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 1175–1191 (2017). https://doi.org/10.1145/3133956.3133982
240. Sav, S., Dmitrienko, A., Papadopoulos, D., Sadeghi, A.-R.: Poseidon: Privacy-preserving federated neural network learning. arXiv preprint arXiv:2009.00349(2020)
241. European Commission High-Level Expert Group on Artificial Intelligence: Ethics Guidelines for Trustworthy AI. Accessed: 2026-01-14. https://digital-strategy.ec.europa.eu/en/library/ethics-guidelines-trustworthy-ai
242. Barredo Arrieta, A., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., Chatila, R., Herrera, F.: Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges. Information Fusion **58**, 82–115 (2020)
243. Dean, J., Barroso, L.A.: The tail at scale. Commun. ACM **56**(2), 74–80 (2013). https://doi.org/10.1145/2408776.2408794
244. Doshi-Velez, F., Kim, B.: Towards a rigorous science of interpretable machine learning. arXiv preprint arXiv:1702.08608 (2017)
245. Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., Pedreschi, D.: A survey of methods for explaining black box models. ACM Comput. Surv. **51**(5), 93–19342 (2018). https://doi.org/10.1145/3236009
246. Glymour, M., Glymour, C., Jewell, N.: Causal Inference in Statistics: A Primer. Wiley, Hoboken, NJ, USA (2016)
247. Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., Mané, D.: Concrete problems in ai safety. arXiv preprint arXiv:1606.06565 (2016)
248. Li, B., Peng, X., Xiang, Q., Wang, H., Xie, T., Sun, J., Liu, X.: Enjoy your observability: An industrial survey of microservice tracing and analysis. Empir. Softw. Eng. **27**(1), 25 (2022)
249. OpenTelemetry: OpenTelemetry Specification. https://opentelemetry.io/. Accessed: 2024-07-22 (2023)

Luís M. Barata was born in Castelo Branco, Portugal in 1977. Graduated in Computer Engineering at the Superior School of Technology, Polytechnic Institute of Castelo Branco, and MSc in Software Development and Interactive Systems in the same institution with a master thesis named "Informatics at the Service of the Elderly - an emerging theory," combining the use of IT by the elderly using a Grounded Theory approach. Currently pursuing a PhD in computer engineering at the University of Beira Interior Department of Computer Science in Covilhã, Portugal, his research focus has shifted to microservices architecture, capturing anomalies and determining their cause. Topics such as predictive algorithms, machine learning, and federated learning have recently attracted considerable attention. From 2015, he collaborated with the Superior School of Technology, Polytechnic Institute of Castelo Branco, as an Assistant Professor in the Computer Science degree course. The CTO of BEIRANET - Soluções Informáticas and CEO of BEIRANET II - Serviços Empresariais – has developed several projects since 2002 in web content management platforms and systems programming in the web environment. Business Manager in the WEB area, Software Development, and Business Management Software. WEB programming enthusiasts have developed various WEB systems in JavaScript/VBScript/ASP/SQLServer and PHP/MySQL, including a Document Management System for some companies of the Águas de Portugal group. Eng. Luis Barata is currently a Level 2 Engineer by the Portuguese Order of Engineers in the College of Computer Science and holds the title of Specialist in Development of Information Systems (Computer Science), awarded by Polytechnic Institute of Castelo Branco, Polytechnic Institute of Santarém, Polytechnic Institute of Guarda, Portuguese Order of Engineers (OE) and Portuguese Information Systems Association (APSI).

Sérgio Sequeira was born in Castelo Branco, Portugal in 1979. Graduated in Computer Engineering at Superior School of Technology, Polytechnic Institute of Castelo Branco. Currently studying computer engineering doctoral program at the University of Beira Interior in Covilhã, Portugal. Currently, the focus of research is in the area of security and IoT. Topics such as Smart Cities, Embedding Security by Design, Cloud, and IoT Security. From 2015, he taught computer science at the Superior School of Technology, Polytechnic Institute of Castelo Branco. CEO in BEIRANET - Soluções Informáticas and BEIRANET II - Serviços Empresariais. Since 2002, he has been developing network and security implementation projects, ERP implementation projects, systems, and networks administration. He is the Business Manager for the Areas of Network and Information Systems and for the area of software for the Social-IPSS market. He has several certifications from the most renowned equipment and software manufacturers, such as Huawei, Acronis, Sage, TP-Link, F3M, Cisco, VM-Ware, Veeam.).

Eurico Lopes Computer Engineering, IST, Lisbon. MSc Computer Science, Essex University, UK. PhD Information Management, Leeds Metropolitan University, UK. Professor at IPCB, Portugal. Whilst teaching and researching, Eurico has been working as a consultant with local enterprises. Since the 1990s, Eurico has developed work as a researcher in computer engineering and information systems. Prior to this, Eurico assisted the banking sector and the industry as an engineer, programmer, systems analyst, and trainer, and latterly as a project leader. Whilst teaching and researching, Eurico has established various collaborations, including several major projects with local enterprises, involving technology transfer across the organization for the development and implementation of information systems. For many years Eurico has been a member of Ordem dos Engenheiros, Senior Member in the IEEE and ACM; also working with CCISP (Conselho Coordenador Institutos Superior Politécnicos) as a consultant on ICT and network projects, including work on PRODEP I & II. Eurico Lopes was convened as an observer in the users panel on FCCN (Fundação Computação Cientifica Nacional) and as a member of the Installation Committee Technological and Management School in Castelo Branco and Idanha-a-Nova. Since April 2013, Eurico has been appointed as an Expert at the Agencia de Avaliação e Acreditação do Ensino Superior (Agency for Assessment and Accreditation of Higher Education - A3ES). Research Interests: Project Management, Business Analytics, Decision Support Systems, Research Methodology and Systems Thinking.

Pedro R. M. Inácio is an associate professor of the Department of Computer Science at the University of Beira Interior (UBI), which he joined in 2010. Lectures subjects related with information assurance and (cyber)security, to graduate and undergraduate courses, namely to the B.Sc., M.Sc. and Ph.D. programmes in Computer Science and Engineering. He holds a 5-year B.Sc. degree in Mathematics/Computer Science and a Ph.D. degree in Computer Science and Engineering, obtained from UBI, Portugal, in 2005 and 2009 respectively. The Ph.D. work was performed in the enterprise environment of Nokia Siemens Networks Portugal S.A., through a Ph.D. grant from the Portuguese Foundation for Science and Technology. He is an IEEE senior member, an ACM professional member and a researcher of the Instituto de Telecomunicações (IT). His main research topics are information assurance and security, computer based simulation, and network traffic monitoring, analysis and classification. He frequently reviews papers for IEEE, Springer, Wiley and Elsevier journals. He is a member of the Technical Program Committees of flagship national and international workshops and conferences, such as ACM SAC, IEEE NCA, IFIPSEC or ARES.

Mário M. Freire received a five-year BSc degree in Electrical Engineering and a MSc degree in Systems and Automation in 1992 and 1994, respectively, from the University of Coimbra, Portugal. He received a PhD degree in Electrical Engineering in 2000 and a Habilitation title in Computer Science in 2007 from the University of Beira Interior (UBI), Portugal. He is a full professor of Computer Science at UBI, which he joined in October 1994. He began his research activities in October 1991 at the University of Coimbra, having worked within the European Project EEC RACE 2011 TRAVEL, within which he carried out a one-month internship in April 1993 at the ALCATEL SEL AG (now Nokia Networks) Research Center in Stuttgart, Germany. His main research interests fall within the area of computer systems and networks, including network and systems virtualization, cloud and edge computing, computer security and privacy, and applications of AI in computer systems and networks. He is the coauthor of six international patents, co-editor of eight books published in the Springer LNCS book series, and co-author of more than 130 papers in international journals and conferences. He serves as a member of the editorial board of the ACM SIGAPP Applied Computing Review, serves as an associate editor of the Wiley Security and Privacy Journal and of the Wiley International Journal of Communication Systems, and served as editor of IEEE Communications Surveys and Tutorials in 2007–2011. He served on the technical program committee for several IEEE international conferences and co-chaired the Computer Networking track at ACM SAC from 2007 to 2026. Dr. Mário Freire is a chartered engineer by the Portuguese Engineering Association and is a member of the IEEE Computer Society and the Association for Computing Machinery.